

문제 6

[Kaggle 형] train_prob.csv로 문제 failure 예측하는 모델을 만들고,

test_prob.csv에 대한 failure가 1일 확률 예측하여 다음과 같은 형식의 answer6.csv를 만들어라.

측정 지표는 AUC(area under of ROC curve)이다. id 는 테스트 케이스의 id 이고, failure에는 failure가 1이 될 확률이다.

id,failure

16115, 0.1

16116, 0.2

강사: 멀티캠퍼스 강선구(sunku0316.kang@multicampus.com, sun9sun9@gmail.com)

```
In [1]: # 실행 환경 확인

import pandas as pd
import numpy as np
import sklearn
import scipy
import statsmodels
import mlxtend
import sys
import xgboost as xgb

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

print(sys.version)
for i in [pd, np, sklearn, scipy, mlxtend, statsmodels, xgb]:
    print(i.__name__, i.__version__)
```

```
3.7.4 (tags/v3.7.4:e09359112e, Jul 8 2019, 20:34:20) [MSC v.1916 64 bit (AMD64)]
pandas 0.25.1
numpy 1.18.5
sklearn 0.21.3
scipy 1.5.2
mlxtend 0.15.0.0
statsmodels 0.11.1
xgboost 0.80
```

```
In [2]: df_train = pd.read_csv('train_prob.csv', index_col='id')
df_test = pd.read_csv('test_prob.csv', index_col='id')
df_ans = pd.read_csv('test_prob_ans.csv', index_col='id')
```

```
In [3]: df_train
```

```
Out[3]:
```

	product_code	loading	attribute_0	attribute_1	attribute_2	attribute_3	measurement_0	measurement_1	measurement_2	measurement_3	...	m
id												
0	A	80.10	material_7	material_8	9	5	7	8	4	18.040	...	
1	A	84.89	material_7	material_8	9	5	14	3	3	18.213	...	
2	A	82.43	material_7	material_8	9	5	12	1	5	18.057	...	
3	A	101.07	material_7	material_8	9	5	13	2	6	17.295	...	
4	A	188.06	material_7	material_8	9	5	9	2	8	19.346	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
26565	E	158.95	material_7	material_6	6	9	6	16	4	16.301	...	
26566	E	146.02	material_7	material_6	6	9	10	12	8	17.543	...	
26567	E	115.62	material_7	material_6	6	9	1	10	1	15.670	...	
26568	E	106.38	material_7	material_6	6	9	2	9	4	18.059	...	
26569	E	131.20	material_7	material_6	6	9	6	19	1	18.034	...	

21458 rows × 25 columns

```
In [4]: df_train.columns
```

```
Out[4]: Index(['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2',
              'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2',
              'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6',
              'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10',
              'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14',
              'measurement_15', 'measurement_16', 'measurement_17', 'failure'],
              dtype='object')
```

```
In [5]: df_ans
```

```
Out[5]:
```

	failure
id	
16115	1
16116	0
16117	0
16118	1
16119	0
...	...
21222	0
21223	0
21224	0
21225	1
21226	0

5112 rows × 1 columns

```
In [6]: '''
```

이 코드는 `df\_train`과 `df\_test`라는 두 개의 DataFrame에 대해 작업을 수행하고 있습니다.

코드의 목적은 각 DataFrame에서 특정 열의 결측값(NaN)을 찾고, 해당 위치에 새로운 열을 추가하여 결측값이 있는지 여부를 표시하는 것입니다.

1. 특정 열 선택 및 결측값 확인

이 부분은 `measurement\_3`과 `measurement\_5`라는 두 개의 열 이름을 리스트 형태로 생성합니다.
구체적으로, `[ 'measurement\_3', 'measurement\_5' ]`와 동일한 결과를 얻습니다.

2. 결측값 확인

```
df_train[['measurement_{}'.format(i) for i in [3, 5]]].isna()
```

이 코드는 `df\_train` DataFrame에서 `measurement\_3`과 `measurement\_5` 열을 선택하고, 각 셀에 대해 결측값 (NaN)이 있는지 확인합니다.
`.isna()` 메서드는 각 셀에 결측값이 있는 경우 `True`, 그렇지 않은 경우 `False`를 반환하는 동일한 크기의 DataFrame을 반환합니다.

3. 새로운 열에 결측값 여부 저장

```
df_train[['na_1', 'na_2']] = ...
```

이 코드는 앞서 얻은 결측값 여부를 표시하는 DataFrame을 `df\_train`에 새로운 두 개의 열(`na\_1` 및 `na\_2`)로 추가합니다.
`na\_1` 열은 `measurement\_3` 열의 결측값 여부를, `na\_2` 열은 `measurement\_5` 열의 결측값 여부를 저장합니다.

전체 코드 설명

위의 설명을 종합하여 코드를 이해하면 다음과 같습니다:

df_train의 'measurement_3'과 'measurement_5' 열의 결측값 여부를 각각 'na_1'과 'na_2' 열에 저장

df_test의 'measurement_3'과 'measurement_5' 열의 결측값 여부를 각각 'na_1'과 'na_2' 열에 저장

- `df\_train[['measurement\_3', 'measurement\_5']]` 부분에서 `df\_train` DataFrame에서 `measurement\_3`과 `measurement\_5` 열을 선택함

- `.isna()` 메서드를 사용하여 각 셀에 대해 결측값 여부를 확인합니다.

- `[ ['na\_1', 'na\_2']]`로 새로운 열을 지정하여 결측값 여부를 `True` 또는 `False`로 표시합니다.

결과적으로, `df\_train`과 `df\_test` DataFrame에는 `measurement\_3`과 `measurement\_5` 열의
결측값 여부를 나타내는 `na\_1`과 `na\_2`라는 새로운 열이 추가됩니다.

```
...
```

```
df_train[['na_1', 'na_2']] = df_train[['measurement_{}'.format(i) for i in [3, 5]]].isna()
```

```
df_test[['na_1', 'na_2']] = df_test[['measurement_{}'.format(i) for i in [3, 5]]].isna()
```

```
In [7]: df_train['na_1'].value_counts()
```

```
Out[7]: False    21146
        True      312
        Name: na_1, dtype: int64
```

```
In [8]: df_train['na_2'].value_counts()
```

```
Out[8]: False    20893
        True      565
        Name: na_2, dtype: int64
```

```
In [9]: df_test['na_1'].value_counts()
```

```
Out[9]: False    5043
        True      69
        Name: na_1, dtype: int64
```

```
In [10]: df_test['na_2'].value_counts()
```

```
Out[10]: False    5001
         True      111
         Name: na_2, dtype: int64
```

```
In [12]: '''
`IterativeImputer`는 scikit-learn 라이브러리에서 제공하는 클래스 중 하나로, 결측값을 다루기 위해 사용됩니다.
이 클래스는 다중 대체법(Multiple Imputation by Chained Equations, MICE)을 사용하여 결측값을 예측합니다.
이는 각 특성을 반복적으로 모델링하여 다른 특성들로부터 결측값을 예측하는 방식입니다.
```

주요 인자 (Parameters)

- **estimator**: 기본값은 ``BayesianRidge()``. 결측값을 대체하는 데 사용되는 추정기(estimator)입니다. ``scikit-learn``의 다른 추정기로 대체할 수 있습니다.
- **missing_values**: 기본값은 ``np.nan``. 결측값으로 간주할 값입니다.
- **max_iter**: 기본값은 `10`. 반복할 최대 횟수입니다.
- **random_state**: 기본값은 ``None``. 랜덤 상태를 설정하여 결과의 재현성을 보장할 수 있습니다.
- **initial_strategy**: 기본값은 ``'mean'``. 초기 결측값을 대체하는 전략으로, ``'mean'``, ``'median'``, ``'most_frequent'``, ``'constant'`` 중 선택할 수 있습니다.
- **imputation_order**: 기본값은 ``'ascending'``. 결측값을 대체하는 순서입니다. ``'ascending'``, ``'descending'``, ``'roman'``, ``'arabic'``, ``'random'`` 중 선택할 수 있습니다.
- **min_value**: 기본값은 ``-np.inf``. 대체값의 최소값을 설정할 수 있습니다.
- **max_value**: 기본값은 ``np.inf``. 대체값의 최대값을 설정할 수 있습니다.
- **verbose**: 기본값은 `0`. 진행 상황을 출력할지 여부를 설정합니다.
- **tol**: 기본값은 `1e-3`. 수렴 기준으로 사용됩니다.

리턴값

- ``IterativeImputer`` 클래스의 객체는 다음과 같은 주요 메서드와 속성을 가집니다.

주요 속성 (Attributes)

- **initial_imputer_**: 초기 결측값 대체를 위해 사용되는 `SimpleImputer` 객체.
- **imputation_sequence_**: 각 반복마다 사용된 대체 전략의 순서.
- **n_iter_**: 수행된 반복 횟수.

주요 메서드 (Methods)

```
- fit(X, y=None): 훈련 데이터 `X`를 사용하여 대체 모델을 학습합니다.
- transform(X): 학습된 모델을 사용하여 `X`의 결측값을 대체합니다.
- fit_transform(X, y=None): `fit`과 `transform`을 결합하여 `X`를 학습하고 대체합니다.
- get_params(deep=True): 추정기의 매개변수를 딕셔너리로 반환합니다.
- set_params(params): 추정기의 매개변수를 설정합니다.
'''
```

```
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
```

```
# 데이터 생성
```

```
X = np.array([[1, 2, np.nan, 4],
              [5, np.nan, 6, 8],
              [9, 10, 11, 12],
              [np.nan, 14, 15, 16]])
```

```
print(X)
```

```
print('-----')
```

```
# IterativeImputer 객체 생성
```

```
imputer = IterativeImputer(max_iter=10, random_state=0)
```

```
# 데이터에 맞게 imputer를 학습하고 변환
```

```
X_imputed = imputer.fit_transform(X)
```

```
print(X_imputed)
```

```
[[ 1.  2. nan  4.]
 [ 5. nan  6.  8.]
 [ 9. 10. 11. 12.]
 [nan 14. 15. 16.]]
```

```
-----
[[ 1.          2.          1.6874879  4.          ]
 [ 5.          5.99922427  6.          8.          ]
 [ 9.          10.         11.         12.          ]
 [13.00070736 14.         15.         16.          ]]
```

```
In [7]: '''
```

```
아래 코드는 결측값 처리를 위해 두 가지 방법을 사용하여 `df_train`과 `df_test` DataFrame의 특정 열을 변형합니다.
각 DataFrame에서 결측값을 다루기 위해 Iterative Imputer와 평균 대체법을 사용합니다.
```

```
# 1. Iterative Imputer를 사용한 결측값 대체
## 1.1 Iterative Imputer 설정
- `IterativeImputer`는 여러 예측 모델을 사용하여 결측값을 대체하는 방법입니다.
- `LinearRegression(fit_intercept=True)`를 예측 모델로 사용합니다. 이는 회귀 분석을 통해 결측값을 예측하는 역할을 합니다.
## 1.2 결측값을 대체할 열 목록 정의
- `measurement_3`부터 `measurement_9`까지와 `measurement_17` 열의 목록을 정의합니다.
- `X_imp`는 이 열들의 리스트입니다.
## 1.3 Iterative Imputer를 사용하여 결측값 대체
- `df_train`과 `df_test` DataFrame을 `product_code`로 그룹화하여 각 그룹에 대해 Iterative Imputer를 적용합니다.
- `imp.fit_transform(x)`를 사용하여 각 그룹의 결측값을 대체합니다.
- 결과를 원래의 인덱스와 열 이름을 유지하면서 새로운 DataFrame으로 변환하여 원래 DataFrame의 해당 열에 할당합니다.

# 2. 평균 대체법을 사용한 결측값 대체
## 2.1 평균 대체법을 사용할 열 목록 정의
- `measurement_10`부터 `measurement_16`까지의 열 목록을 정의합니다.
- `X_mean`는 이 열들의 리스트입니다.
## 2.2 평균 대체법을 사용하여 결측값 대체
- `df_train`과 `df_test` DataFrame을 `product_code`로 그룹화하여 각 그룹에 대해 평균 대체법을 적용합니다.
- `x.fillna(x.mean())`를 사용하여 각 그룹의 결측값을 해당 그룹의 평균값으로 대체합니다.
- `transform` 메서드는 그룹화된 데이터의 원래 인덱스를 유지하면서 변형을 적용합니다.

# 전체 코드 설명
1. Iterative Imputer 설정 및 사용:
    - `measurement_3`부터 `measurement_9`, `measurement_17`까지의 결측값을 IterativeImputer를 사용하여 대체합니다.
    - 각 그룹(`product_code`)별로 Iterative Imputer를 적용하여 결측값을 예측 모델(Linear Regression)로 대체합니다.
2. 평균 대체법 사용:
    - `measurement_10`부터 `measurement_16`까지의 결측값을 각 그룹(`product_code`)의 평균값으로 대체합니다.

코드는 두 가지 결측값 대체 방법을 사용하여 데이터의 결측값을 효과적으로 처리하여 모델 학습에 사용할 수 있도록 합니다.
...
```

```
# 방법 1: product_code 별로 IterativeImputer를 통한 결측치처리를 하고 바로 적용합니다.
```

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

imp = IterativeImputer(
    estimator=LinearRegression(fit_intercept=True),
    random_state=123
)
```

```

X_imp = ['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17']
print('X_imp = ', X_imp)

df_train[X_imp] = df_train.groupby('product_code')[X_imp].apply(
    lambda x: pd.DataFrame(imp.fit_transform(x), index=x.index, columns=x.columns)
)

df_test[X_imp] = df_test.groupby('product_code')[X_imp].apply(
    lambda x: pd.DataFrame(imp.fit_transform(x), index=x.index, columns=x.columns)
)

X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
print('X_mean = ', X_mean)

df_train[X_mean] = df_train.groupby('product_code')[X_mean].transform(
    lambda x: x.fillna(x.mean())
)

df_test[X_mean] = df_test.groupby('product_code')[X_mean].transform(
    lambda x: x.fillna(x.mean())
)

```

```

X_imp = ['measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9',
'measurement_17']
X_mean = ['measurement_10', 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14', 'measurement_15', 'measurem
ent_16']

```

In [8]:

```

'''
코드는 `df_train`과 `df_test` DataFrame에 있는 특정 열들의 결측값을 처리하는 방법을 보여줍니다.
결측값 처리를 위해 두 가지 방법을 사용합니다: Iterative Imputer와 평균 대체법.

# 1. Iterative Imputer를 사용한 결측값 대체
## 1.1 결측값을 대체할 열 목록 정의
- `measurement_3`부터 `measurement_9`까지와 `measurement_17`의 열 목록을 정의합니다.
- `X_imp`는 이 열들의 리스트입니다.
## 1.2 Iterative Imputer 모델 학습
- `df_train`과 `df_test` DataFrame을 결합하여 결측값을 처리할 데이터를 만듭니다.
  이 결합된 DataFrame에서 `X_imp` 열들과 `product_code` 열을 선택합니다.
- 결합된 DataFrame을 `product_code`로 그룹화하여 각 그룹에 대해 Iterative Imputer를 학습시킵니다.
- 각 그룹별로 Iterative Imputer를 생성하고 `X_imp` 열에 대해 학습시킵니다.
## 1.3 학습된 모델을 train 데이터에 적용
- `df_train` DataFrame을 `product_code`로 그룹화하여 각 그룹에 대해 학습된 Iterative Imputer 모델을 적용합니다.

```



```

- 각 그룹별로 `transform` 메서드를 사용하여 `X_imp` 열의 결측값을 대체합니다.
- 결과를 원래의 인덱스와 열 이름을 유지하면서 새로운 DataFrame으로 변환하여 원래 DataFrame의 해당 열에 할당합니다.
## 1.4 학습된 모델을 test 데이터에 적용
- `df_test` DataFrame에 대해서도 동일한 방식으로 학습된 Iterative Imputer 모델을 적용합니다.

# 2. 평균 대체법을 사용한 결측값 대체
## 2.1 평균 대체법을 사용할 열 목록 정의
- `measurement_10`부터 `measurement_16`까지의 열 목록을 정의합니다.
- `X_mean`는 이 열들의 리스트입니다.
## 2.2 평균값 계산
- `df_train`과 `df_test` DataFrame을 결합하여 결측값을 처리할 데이터를 만듭니다.
  이 결합된 DataFrame에서 `X_mean` 열들과 `product_code` 열을 선택합니다.
- 결합된 DataFrame을 `product_code`로 그룹화하여 각 그룹의 `X_mean` 열들에 대한 평균값을 계산합니다.
## 2.3 평균값을 사용하여 결측값 대체 (train 데이터)
- `df_train` DataFrame을 `product_code`로 그룹화하여 각 그룹에 대해 결측값을 해당 그룹의 평균값으로 대체합니다.
- 각 그룹별로 `fillna` 메서드를 사용하여 `X_mean` 열의 결측값을 대체합니다.
- 결과를 원래의 인덱스와 열 이름을 유지하면서 새로운 DataFrame으로 변환하여 원래 DataFrame의 해당 열에 할당합니다.
## 2.4 평균값을 사용하여 결측값 대체 (test 데이터)
- `df_test` DataFrame에 대해서도 동일한 방식으로 결측값을 대체합니다.

# 전체 코드 설명
1. Iterative Imputer 설정 및 사용:
  - `measurement_3`부터 `measurement_9`, `measurement_17`까지의 결측값을 Iterative Imputer를 사용하여 대체합니다.
  - `df_train`과 `df_test`를 결합하여 `product_code`별로 Iterative Imputer 모델을 학습하고,
    학습된 모델을 `df_train`과 `df_test`에 각각 적용합니다.
2. 평균 대체법 사용:
  - `measurement_10`부터 `measurement_16`까지의 결측값을 각 그룹(`product_code`)의 평균값으로 대체합니다.
  - `df_train`과 `df_test`를 결합하여 각 그룹의 평균값을 계산하고, 이를 사용하여 결측값을 대체합니다.
코드는 결측값을 효과적으로 처리하여 모델 학습에 사용할 수 있도록 합니다.
결측값 처리를 위해 두 가지 다른 방법을 사용하여 데이터의 완전성을 보장합니다.
...

# 방법 2 : train / test를 통합하여 처리해봅니다.

# 구문을 사용하여 실험 단계인 모듈을 활성화하고,
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X_imp = ['measurement_{}'.format(i) for i in range(3, 10)] + ['measurement_17']
# train에 등장하지 않은 수준이 있습니다, test를 포함하여 결측치 처리 모델을 만듭니다.

```

```
s_imp = pd.concat([df_train[X_imp + ['product_code']], df_test[X_imp + ['product_code']], axis=0)\
    .groupby('product_code')\
    .apply(lambda x: IterativeImputer(estimator=LinearRegression(), random_state=123).fit(x[X_imp]))

# train에 적용합니다.
df_train[X_imp] = df_train[X_imp + ['product_code']]\
    .groupby('product_code')\
    .apply(lambda x: pd.DataFrame(s_imp.loc[x.name].transform(x[X_imp]), index=x.index, columns=X_imp))

# test에 적용합니다.
df_test[X_imp] = df_test[X_imp + ['product_code']]\
    .groupby('product_code')\
    .apply(lambda x: pd.DataFrame(s_imp.loc[x.name].transform(x[X_imp]), index=x.index, columns=X_imp))

X_mean = ['measurement_{}'.format(i) for i in range(10, 17)]
# 역시 train에 등장하지 않은 수준을 처리하기 위해 합칩니다.
df_mean = pd.concat([df_train[['product_code'] + X_mean], df_test[['product_code'] + X_mean]])\
    .groupby('product_code')[X_mean].agg('mean')

df_train[X_mean] = df_train.groupby('product_code')[X_mean]\
    .apply(lambda x: pd.DataFrame(x.fillna(df_mean.loc[x.name]), index=x.index, columns=x.columns))

df_test[X_mean] = df_test.groupby('product_code')[X_mean]\
    .apply(lambda x: pd.DataFrame(x.fillna(df_mean.loc[x.name]), index=x.index, columns=x.columns))
```

In [9]:

```
...
코드는 `df_train`과 `df_test` DataFrame의 `loading` 열에 있는 결측값을 처리하는 방법을 보여줍니다.
결측값을 각 DataFrame에서 평균값으로 채웁니다.

### 1. `df_train`의 `loading` 열 결측값 채우기
df_train['loading'] = df_train['loading'].fillna(df_train['loading'].mean())
- `df_train['loading']`는 `df_train` DataFrame의 `loading` 열을 선택합니다.
- `.fillna(df_train['loading'].mean())`는 `loading` 열의 결측값을 `df_train['loading']`의 평균값으로 대체합니다.
- `df_train['loading'].mean()`는 `df_train`의 `loading` 열의 평균값을 계산합니다.
- 따라서, `df_train['loading']`의 결측값은 `df_train['loading']`의 평균값으로 채워집니다.

### 2. `df_test`의 `loading` 열 결측값 채우기
df_test['loading'] = df_test['loading'].fillna(df_train['loading'].mean())
- `df_test['loading']`는 `df_test` DataFrame의 `loading` 열을 선택합니다.
- `.fillna(df_train['loading'].mean())`는 `loading` 열의 결측값을 `df_train['loading']`의 평균값으로 대체합니다.
- `df_train['loading'].mean()`는 `df_train`의 `loading` 열의 평균값을 계산합니다.
- 따라서, `df_test['loading']`의 결측값은 `df_train['loading']`의 평균값으로 채워집니다.
```

전체 코드 설명

1. `df\_train`의 `loading` 열 결측값 채우기:

- `df\_train` DataFrame의 `loading` 열에서 결측값을 `df\_train['loading']`의 평균값으로 대체합니다.
- 이를 통해 `df\_train`의 `loading` 열에 더 이상 결측값이 존재하지 않도록 합니다.

2. `df\_test`의 `loading` 열 결측값 채우기:

- `df\_test` DataFrame의 `loading` 열에서 결측값을 `df\_train['loading']`의 평균값으로 대체합니다.
- `df\_test`의 `loading` 열의 결측값을 `df\_train`의 `loading` 열의 평균값으로 채우는 이유는, 모델이 일관된 기준으로 결측값을 처리할 수 있도록 하기 위함입니다. 이는 특히 `df\_test`의 `loading` 열에 결측값이 있는 경우에 유용합니다.

이 코드의 목적은 `df\_train`과 `df\_test`의 `loading` 열에서 결측값을 제거하여 데이터의 완전성을 높이고, 결측값이 모델의 성능에 부정적인 영향을 미치지 않도록 하는 것입니다.

...

```
df_train['loading'] = df_train['loading'].fillna(df_train['loading'].mean())
df_test['loading'] = df_test['loading'].fillna(df_train['loading'].mean())
```

In [10]: `df_train['product_code'].value_counts()`

```
Out[10]: C    5765
         E    5343
         B    5250
         A    5100
         Name: product_code, dtype: int64
```

In [11]: `df_test['product_code'].value_counts()`

```
Out[11]: D    5112
         Name: product_code, dtype: int64
```

In [15]: ...

`GroupKFold`는 `scikit-learn` 라이브러리에서 제공하는 교차 검증 도구 중 하나입니다.

그룹 정보를 사용하여 데이터를 분할하는 데 특화되어 있습니다.

이는 특히 동일한 그룹에 속한 샘플이 항상 같은 훈련 세트 또는 검증 세트에 포함되도록 보장해야 하는 경우에 유용합니다.

주요 인자 (Parameters)

- `n_splits`: 기본값은 5. 교차 검증을 위해 데이터셋을 분할하는 부분 수입니다. 최소값은 2입니다.

리턴값

- `GroupKFold` 객체는 교차 검증 동안 학습 및 테스트 데이터를 생성하는데 사용됩니다. 이는 `split` 메서드를 통해 이루어집니다.

```

### 주요 속성 (Attributes)
- n_splits: 지정된 분할 수를 저장합니다.

### 주요 메서드 (Methods)
- split(X, y=None, groups=None): 주어진 데이터셋 `X`, 목표 값 `y`, 그룹 라벨 `groups`에 대해 학습/검증 인덱스를 생성합니다.
  - X: 학습 데이터.
  - y: 목표 값 (선택 사항).
  - groups: 각 샘플에 대한 그룹 라벨. 이 값이 제공되어야 그룹별로 분할이 가능합니다.
이 메서드는 각 분할마다 학습 세트와 검증 세트의 인덱스를 반환하는 제너레이터를 반환합니다.

### `split` 메서드 동작 원리
`split` 메서드는 다음과 같은 단계를 통해 동작합니다:
1. `groups`를 기준으로 데이터셋을 분할합니다.
2. `n_splits`에 따라 각 분할마다 학습 세트와 검증 세트를 생성합니다.
3. 각 분할마다 학습 세트와 검증 세트의 인덱스를 제너레이터로 반환합니다.

### 요약
`GroupKFold`는 그룹별로 데이터를 분할하는 교차 검증 방법을 제공합니다.
이는 그룹 내 샘플들이 항상 같은 훈련 또는 검증 세트에 포함되도록 하는데 유용합니다.
주요 매개변수로는 `n_splits`가 있으며, `split` 메서드를 사용하여 데이터를 그룹별로 분할합니다.
...

...

아래 예제에서는 `GroupKFold`를 사용하여 데이터를 세 그룹으로 분할합니다.
각 분할에서는 동일한 그룹이 항상 같은 세트(훈련 세트 또는 검증 세트)에 속합니다.
...

import numpy as np
from sklearn.model_selection import GroupKFold

# 데이터 생성
X = np.arange(10).reshape(5, 2)
print(X)
print('-----')
y = np.array([0, 1, 0, 1, 0])
groups = np.array([1, 1, 2, 2, 3])

# GroupKFold 객체 생성
gkf = GroupKFold(n_splits=3)

# 분할
for train_index, test_index in gkf.split(X, y, groups):
    print("TRAIN:", train_index, "TEST:", test_index)

```

```
X_train, X_test = X[train_index], X[test_index]
y_train, y_test = y[train_index], y[test_index]
print("X_train:\n", X_train)
print("X_test:\n", X_test)
print("y_train:", y_train)
print("y_test:", y_test)
print('-----')
```

```

[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]
-----
TRAIN: [0 1 4] TEST: [2 3]
X_train:
[[0 1]
 [2 3]
 [8 9]]
X_test:
[[4 5]
 [6 7]]
y_train: [0 1 0]
y_test: [0 1]
-----
TRAIN: [2 3 4] TEST: [0 1]
X_train:
[[4 5]
 [6 7]
 [8 9]]
X_test:
[[0 1]
 [2 3]]
y_train: [0 1 0]
y_test: [0 1]
-----
TRAIN: [0 1 2 3] TEST: [4]
X_train:
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
X_test:
[[8 9]]
y_train: [0 1 0 1]
y_test: [0]
-----

```

In [17]: '''

코드의 목적은 `GroupKFold`를 사용하여 데이터셋을 그룹화한 뒤, 각 그룹별로 교차 검증을 수행하는 것입니다.

1. GroupKFold 정의:

이 코드는 4-폴드 그룹 교차 검증을 설정합니다. **GroupKFold**는 그룹 단위로 데이터를 나누어 교차 검증을 수행하며, 같은 그룹에 속한 데이터는 항상 같은 훈련 또는 검증 세트에 배치됩니다.

2. 교차 검증 수행:

`gcv.split()` 메서드는 `df_train` 데이터프레임을 4개의 폴드로 나누는 인덱스들을 생성합니다. 여기서 중요한 점은 `groups=df_train['product_code']` 인수를 사용한다는 것입니다. 이는 `'product_code'` 열에 따라 데이터를 그룹화하여 교차 검증을 수행하겠다는 의미입니다. 따라서, 같은 `'product_code'`를 가진 데이터는 항상 같은 폴드에 속하게 됩니다.

3. 훈련 및 검증 인덱스 출력:

각 폴드에 대해 훈련 인덱스 (`train_idx`)와 검증 인덱스 (`valid_idx`)를 생성한 후, 이 인덱스를 이용하여 해당 폴드에 포함된 `'product_code'`의 유니크 값을 출력합니다. 이 출력은 각 폴드에 포함된 그룹(제품 코드)이 어떤 것들인지 확인하는데 유용합니다.

코드 실행 흐름

1. `GroupKFold` 객체 `gcv`가 생성되어, 데이터셋을 4개의 폴드로 나누도록 설정됩니다.
2. `gcv.split()` 메서드를 호출하여, `df_train`을 그룹화한 후 교차 검증을 수행할 수 있도록 훈련 및 검증 인덱스를 생성합니다.
3. 각 폴드마다 훈련 데이터와 검증 데이터에 포함된 `'product_code'`의 유니크 값을 출력합니다.

각 출력은 훈련 세트와 검증 세트에 포함된 `'product_code'` 값들의 배열입니다. 이를 통해 교차 검증이 제대로 이루어졌는지, 즉 각 폴드에 특정 그룹들이 올바르게 분배되었는지를 확인할 수 있습니다.

Test에서는 Train에 등장하지 않은 product_code가 등장합니다. 이러한 특성을 반영하는 검증법입니다.

```
from sklearn.model_selection import GroupKFold
```

만일 이러한 기준이 되는 범주에 범주수가 충분하여,

n_splits = 5, 10개 등으로 지정해도 불균형하지 않다면 임의성을 가지고 n_splits를 지정할 수 있습니다.

gcv = GroupKFold(4) # n_splits 4를 설정한 건 train의 케이스가 4개라서 입니다.

```
print(df_train['product_code'].nunique())
print('-----')
print(df_train['product_code'].value_counts())
print('-----')
```

```
for train_idx, valid_idx in gcv.split(df_train, groups=df_train['product_code']):
    print(
        df_train.iloc[train_idx]['product_code'].unique(),
        df_train.iloc[valid_idx]['product_code'].unique()
    )
```

```

4
-----
C    5765
E    5343
B    5250
A    5100
Name: product_code, dtype: int64
-----
['A' 'B' 'E'] ['C']
['A' 'B' 'C'] ['E']
['A' 'C' 'E'] ['B']
['B' 'C' 'E'] ['A']

```

In [13]: `Image('2.png')`

Out[13]: 일반 문제에서 얻은 힌트를 정리해봅니다.

- log 자연 로그 함수를 적용하여 파생 변수 `loading_log`
- `LogisticRegression X_sfs_best = ['loading', 'measurement_1', 'measurement_4', 'measurement_14', 'measurement_17', 'na_1'] 0.5838326230092876`
- `LinearDiscriminantAnalysis() measurement_0 ~ 17 ?`
- `PCA() measurement_0 ~ 17 , n_components=7 0.581757510516433`
- `RandomForestClassifier(random_state=123) 'n_estimators': 15, 'max_depth': 7, 'min_samples_split': 512} 0.5745226991354744`

Kaggle형 풀이 단계

Step 1: 검증 방법을 정하고, 검증 루틴을 만듭니다.

Step 2: Baseline 모델을 만듭니다.

Step 3: 모델 선택 루틴을 만듭니다.

Step 4: 모델 개선 작업을 합니다.

Step1: 검증 방법을 정하고, 검증 루틴을 만듭니다.


```
In [22]: '''
`cross_validate` 함수는 scikit-learn에서 제공하는 함수로, 모델을 교차 검증하는데 사용됩니다.
이 함수는 모델 성능을 평가하기 위해 여러 교차 검증 반복에서 여러 평가 지표를 계산합니다.

### 주요 인자 (Parameters)
- estimator: 학습할 모델 객체. 이 객체는 `fit` 메서드를 가지고 있어야 합니다.
- X: (array-like 또는 sparse matrix) 학습 데이터.
- y: (array-like, optional) 목표 값.
- groups: (array-like, optional) 그룹 라벨. `GroupKFold` 등 그룹 기반 분할을 사용할 때 필요합니다.
- scoring: (string, callable, list/tuple, dict, or None, optional) 평가 지표. 여러 평가 지표를 사용할 수 있습니다.
    기본값은 `None`으로, 이 경우 `estimator`의 `score` 메서드를 사용합니다.
- cv: (int, cross-validation generator, or iterable, optional) 교차 검증 전략. 기본값은 5입니다.
    숫자를 입력하면 K-Fold를 사용합니다. 또한 `StratifiedKFold`, `GroupKFold` 등 다른 교차 검증 생성기를 사용할 수 있습니다.
- n_jobs: (int, optional) 병렬 작업 수. `-1`로 설정하면 모든 CPU 코어를 사용합니다.
- verbose: (int, optional) 진행 상태 출력을 제어합니다.
- fit_params: (dict, optional) `estimator`의 `fit` 메서드에 전달할 매개변수입니다.
- pre_dispatch: (int or string, optional) 병렬 작업 시 할당할 작업 수를 제어합니다.
- return_train_score: (bool, optional) 훈련 세트에서의 점수도 반환할지 여부를 결정합니다. 기본값은 `False`입니다.

### 리턴값 (Returns)
- scores: (dict) 평가 지표를 포함한 결과 딕셔너리. 키는 각 지표의 이름이고, 값은 각 분할에서의 점수입니다.
    기본적으로 테스트 점수는 `test_<scorer_name>` 형식으로 반환됩니다.
    `return_train_score=True`일 경우 훈련 점수도 `train_<scorer_name>` 형식으로 반환됩니다.

### 주요 속성 (Attributes)
`cross_validate` 함수 자체는 속성을 가지지 않지만, 반환된 딕셔너리는 다음과 같은 속성을 포함합니다:
- `fit_time`: 각 분할에서의 모델 학습 시간.
- `score_time`: 각 분할에서의 모델 평가 시간.
- `test_score`: 각 분할에서의 테스트 점수. 여러 평가 지표를 사용하는 경우, `test_<scorer_name>` 형식으로 반환됩니다.
- `train_score`: `return_train_score=True`인 경우, 각 분할에서의 훈련 점수.
    여러 평가 지표를 사용하는 경우, `train_<scorer_name>` 형식으로 반환됩니다.

### 요약
`cross_validate` 함수는 모델의 교차 검증을 수행하고, 다양한 평가 지표에 대해 결과를 제공합니다.
이 함수는 여러 평가 지표를 지원하며, 훈련 및 테스트 점수를 모두 반환할 수 있습니다.
주요 인자로는 `estimator`, `X`, `y`, `scoring`, `cv` 등이 있으며, 반환값은 각 분할에서의 점수를 포함하는 딕셔너리입니다.
'''

...

아래 예제에서는 `LogisticRegression` 모델을 사용하여 `iris` 데이터셋에 대해 교차 검증을 수행합니다.
`accuracy`와 `precision_macro` 두 가지 평가 지표를 사용하며, 훈련 점수도 반환하도록 설정합니다.
```

```

'''
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate
from sklearn.metrics import make_scorer, accuracy_score, precision_score

# 데이터 로드
iris = load_iris()
X, y = iris.data, iris.target

# 모델 생성
model = LogisticRegression(max_iter=200)

# 평가 지표 설정
scoring = {'accuracy': 'accuracy',
           'precision_macro': make_scorer(precision_score, average='macro')}

# 교차 검증
results = cross_validate(model, X, y, scoring=scoring, cv=5, return_train_score=True)

print("Test accuracy:", results['test_accuracy'])
print("Test precision:", results['test_precision_macro'])
print("Train accuracy:", results['train_accuracy'])
print("Train precision:", results['train_precision_macro'])
print("Fit time:", results['fit_time'])
print("Score time:", results['score_time'])

```

```

Test accuracy: [1.          0.96666667 0.93333333 0.9          1.          ]
Test precision: [1.          0.96969697 0.93333333 0.92307692 1.          ]
Train accuracy: [0.95        0.96666667 0.96666667 0.975         0.95833333]
Train precision: [0.95286195 0.96741855 0.96969697 0.97519283 0.96296296]
Fit time: [0.0010004 0.          0.          0.          ]
Score time: [0.00099993 0.          0.00099993 0.00099969 0.          ]

```

In [23]:

```

'''
코드의 목적은 여러 모델을 평가하고 최적의 모델을 선택하여 예측 결과를 파일로 저장하는 것입니다.

```

```

### 초기 설정

```

1. 히스토리 리스트 초기화 : 모델 평가 결과를 저장할 빈 리스트 `s\_hist`를 초기화합니다.
2. GroupKFold 정의 : 4-폴드 그룹 교차 검증을 설정합니다.
 이는 `df\_train` 데이터프레임을 4개의 폴드로 나누어 교차 검증을 수행할 때,
 같은 `product\_code`를 가진 데이터는 항상 같은 폴드에 속하게 합니다.

3. 피처 리스트 설정 : `df\_test` 데이터프레임의 모든 열 이름을 리스트로 저장하여, 이를 모델 학습과 예측에 사용할 피처로 지정합니다.

모델 평가 함수

4. `eval\_model` 함수 정의 : `model\_name` (모델 이름)과 `clf` (모델 객체)를 인수로 받아, 주어진 모델을 평가하는 함수입니다.

5. 모델 교차 검증 수행 : `cross\_validate` 함수를 사용하여 모델을 교차 검증합니다.

`df\_train[X\_all]`은 학습에 사용할 피처이고, `df\_train['failure']`는 타겟 레이블입니다.

그룹은 `df\_train['product\_code']`로 설정되어, 같은 제품 코드를 가진 데이터가 항상 같은 폴드에 속하게 합니다.

평가 지표로 ROC AUC를 사용합니다.

6. 결과 포매팅 및 출력 : 검증 세트와 훈련 세트의 평균 점수와 표준 편차를 계산하여 포매팅된 문자열로 만듭니다.

이 문자열은 교차 검증 결과를 요약합니다.

7. 결과 저장 및 출력:

평가 결과를 `s\_hist` 리스트에 저장하고, 데이터프레임으로 변환하여 각 모델의 마지막 결과를 출력합니다.

모델 선택 함수

8. `select\_model` 함수 정의 : 주어진 모델을 학습시키고 예측 결과를 저장하는 함수입니다.

9. 모델 학습 : 전체 학습 데이터를 사용하여 모델을 학습시킵니다.

10. 예측 수행 : 테스트 데이터에 대한 예측을 수행합니다. `predict\_proba` 메서드는 클래스별 확률을 반환하며, 여기서는 긍정 클래스(실패)의 확률을 선택합니다.

11. 결과 저장 : 예측 결과를 데이터프레임으로 만들어 CSV 파일로 저장합니다. 파일 이름은 'answer6.csv'입니다.

12. 예측 결과 반환 : 예측 결과를 반환합니다.

요약

- `eval\_model` 함수는 주어진 모델을 교차 검증하여 성능을 평가하고 결과를 출력 및 저장합니다.

- `select\_model` 함수는 주어진 모델을 전체 데이터로 학습시켜 예측을 수행하고 결과를 파일로 저장합니다.

...

```
from sklearn.model_selection import GroupKFold
from sklearn.model_selection import cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score
```

```
s_hist = list()
```

```
gcv = GroupKFold(4)
```

```
print('type(df_test.columns) = ', type(df_test.columns))
```

```
print('-----')
```

```
X_all = df_test.columns.tolist()
```

```

print('X_all = ', X_all)
print('-----')

def eval_model(model_name, clf):
    result = cross_validate(
        clf, df_train[X_all], df_train['failure'], groups=df_train['product_code'],
        cv=gcv, return_train_score=True, scoring='roc_auc'
    )

    s = 'Valid: {:.5f}±{:.5f}, V.Train: {:.5f}±{:.5f}'.format(
        np.mean(result['test_score']), np.std(result['test_score']),
        np.mean(result['train_score']), np.std(result['train_score'])
    )
    print(s)
    s_hist.append(pd.Series({'model': model_name, 'result': s}))
    display(pd.DataFrame(s_hist).groupby('model').last())

def select_model(clf):
    clf.fit(df_train[X_all], df_train['failure'])
    prd = clf.predict_proba(df_test[X_all])[:, 1]
    pd.DataFrame({
        'id': df_test.index,
        'failure': prd
    }).to_csv('answer6.csv', index=None)
    return prd

type(df_test.columns) = <class 'pandas.core.indexes.base.Index'>
-----
X_all = ['product_code', 'loading', 'attribute_0', 'attribute_1', 'attribute_2', 'attribute_3', 'measurement_0', 'measurement_1', 'measurement_2', 'measurement_3', 'measurement_4', 'measurement_5', 'measurement_6', 'measurement_7', 'measurement_8', 'measurement_9', 'measurement_10', 'measurement_11', 'measurement_12', 'measurement_13', 'measurement_14', 'measurement_15', 'measurement_16', 'measurement_17', 'na_1', 'na_2']
-----

```

Step2: Baseline 모델을 만듭니다

```

In [24]: '''
`ColumnTransformer`는 scikit-learn 라이브러리에서 제공하는 클래스 중 하나로,
서로 다른 특성(열)에 대해 서로 다른 전처리 과정을 적용할 수 있게 해줍니다.
이는 다양한 데이터 전처리 작업을 결합하여 효율적으로 수행할 수 있게 합니다.

```

주요 인자 (Parameters)

- `transformers`: (list of tuples) 각 변환기와 그 변환기를 적용할 열을 지정하는 튜플의 리스트입니다. 각 튜플은 다음과 같이 구성됩니다:
 - ``name`` (string): 변환기의 이름.
 - ``transformer`` (estimator or 'drop' or 'passthrough'): 열에 적용할 변환기. 'drop'은 해당 열을 삭제하고, 'passthrough'는 변환 없이 그대로 두는 옵션입니다.
 - ``columns`` (string, array-like of string, int, array-like of int, slice or boolean mask array): 변환기를 적용할 열.
- `remainder`: (string, default='drop') 지정되지 않은 열에 대한 처리 방법입니다. 'drop'이면 해당 열을 삭제하고, 'passthrough'이면 변환 없이 그대로 둡니다.
- `sparse_threshold`: (float, default=0.3) 결과가 희소 행렬이 될지 밀집 행렬이 될지 결정하는 임계값입니다. 변환 결과에서 희소 행렬의 비율이 이 값보다 크면 결과는 희소 행렬이 됩니다.
- `n_jobs`: (int or None, default=None) 병렬 처리할 작업 수를 지정합니다. ``-1``은 모든 CPU 코어를 사용합니다.
- `transformer_weights`: (dict, optional) 각 변환기의 가중치를 지정합니다. 키는 변환기 이름, 값은 가중치입니다.
- `verbose`: (bool, default=False) 진행 상황을 출력할지 여부를 결정합니다.

리턴값 (Returns)

- ``ColumnTransformer`` 객체는 변환된 데이터를 반환합니다. ``fit``, ``transform``, ``fit_transform`` 메서드를 통해 데이터를 변환할 수 있습니다.

주요 속성 (Attributes)

- `transformers_`: (list of tuples) 변환기 목록. 이 목록은 ``transformers`` 인자의 튜플 리스트와 유사하지만 각 변환기의 적합 상태를 포함합니다.
- `named_transformers_`: (Bunch) 변환기 이름을 키로, 변환기 객체를 값으로 하는 딕셔너리와 유사한 객체입니다.
- `sparse_output_`: (boolean) 변환 결과가 희소 행렬인지 밀집 행렬인지 여부를 나타냅니다.

주요 메서드 (Methods)

- `fit(X, y=None)`: 데이터 ``X``에 대해 각 변환기를 적합시킵니다.
- `fit_transform(X, y=None)`: 데이터 ``X``에 대해 각 변환기를 적합시키고, 변환된 데이터를 반환합니다.
- `transform(X)`: 각 변환기를 사용하여 데이터 ``X``를 변환합니다.
- `inverse_transform(Xt)`: 변환된 데이터 ``Xt``를 원래 형태로 되돌립니다.
- `get_feature_names_out(input_features=None)`: 변환 후의 특성 이름을 반환합니다.
- `set_params(params)`: 변환기의 매개변수를 설정합니다.
- `get_params(deep=True)`: 변환기의 매개변수를 딕셔너리로 반환합니다.

요약

``ColumnTransformer``는 서로 다른 특성에 대해 서로 다른 전처리 과정을 결합하여 적용할 수 있게 해주는 매우 유용한 도구입니다. 주요 인자로 변환기 목록(``transformers``), 지정되지 않은 열에 대한 처리 방법(``remainder``) 등이 있으며, 반환값은 변환된 데이터를 포함합니다.

주요 속성으로는 변환기 목록(``transformers_``), 변환기 이름을 키로 하는 딕셔너리(``named_transformers_``) 등이 있습니다.

주요 메서드로는 ``fit``, ``fit_transform``, ``transform``, ``inverse_transform`` 등이 있습니다.

...

...

아래 예제에서는 ``ColumnTransformer``를 사용하여 서로 다른 특성에 대해 다른 전처리 과정을 적용합니다.

```
`numeric_feature`와 `another_numeric_feature`에 대해 평균 대체와 표준화를 적용하고,  
`categorical_feature`에 대해 가장 빈번한 값으로 대체하고 원-핫 인코딩을 적용합니다.  
...  
  
import numpy as np  
import pandas as pd  
from sklearn.compose import ColumnTransformer  
from sklearn.preprocessing import StandardScaler, OneHotEncoder  
from sklearn.impute import SimpleImputer  
from sklearn.pipeline import Pipeline  
  
# 데이터 생성  
data = pd.DataFrame({  
    'numeric_feature': [1, 2, np.nan, 4, 5],  
    'categorical_feature': ['A', 'B', 'A', np.nan, 'B'],  
    'another_numeric_feature': [10, np.nan, 30, 40, 50]  
})  
  
print(data)  
print('-----')  
  
# 전처리 파이프라인 정의  
numeric_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='mean')),  
    ('scaler', StandardScaler())  
])  
  
categorical_transformer = Pipeline(steps=[  
    ('imputer', SimpleImputer(strategy='most_frequent')),  
    ('onehot', OneHotEncoder())  
])  
  
# ColumnTransformer 정의  
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', numeric_transformer, ['numeric_feature', 'another_numeric_feature']),  
        ('cat', categorical_transformer, ['categorical_feature'])  
    ]  
)  
  
# 데이터 변환  
transformed_data = preprocessor.fit_transform(data)
```

```
print(transformed_data)
```

	numeric_feature	categorical_feature	another_numeric_feature
0	1.0	A	10.0
1	2.0	B	NaN
2	NaN	A	30.0
3	4.0	NaN	40.0
4	5.0	B	50.0

```
-----
[[-1.41421356 -1.70084013  1.          0.          ]
 [-0.70710678  0.          0.          1.          ]
 [ 0.          -0.18898224  1.          0.          ]
 [ 0.70710678  0.56694671  1.          0.          ]
 [ 1.41421356  1.32287566  0.          1.          ]]
```

In [15]:

```
'''
코드는 데이터 전처리와 모델 학습을 결합하여 교차 검증을 통해 모델 성능을 평가하는 과정을 포함하고 있습니다.

### 전처리와 모델 정의
1. ColumnTransformer 정의:
    - StandardScaler 적용:
        - `StandardScaler()`를 사용하여 ['loading', 'measurement_1', 'measurement_4', 'measurement_14', 'measurement_17'] 열을 표준화
        표준화는 각 열의 평균을 0으로, 표준편차를 1로 맞추는 작업입니다.
    - Passthrough 적용:
        - `passthrough` 옵션을 사용하여 `[ 'na_1' ]` 열은 원래의 값을 그대로 유지하여 전달합니다.
2. Logistic Regression 모델 정의:
    - Pipeline 구성:
        - `ColumnTransformer`와 `LogisticRegression`을 결합하여 파이프라인을 만듭니다.
        이렇게 하면 전처리와 모델 학습이 일관된 방식으로 수행됩니다.
    - Logistic Regression:
        - `LogisticRegression` 모델을 사용하며, `solver='lbfgs'`를 지정하여 최적화 알고리즘으로 `lbfgs`를 사용합니다.

### 모델 평가 함수 호출
3. 모델 평가 함수 호출:
    - `eval_model` 함수를 호출하여 'baseline'이라는 이름으로 정의한 `clf_lr` 모델을 평가합니다.
    - `eval_model` 함수는 이전에 정의한 대로 모델을 교차 검증하고 결과를 출력 및 저장합니다.

### `eval_model` 함수의 내부 동작
`eval_model` 함수는 주어진 모델을 교차 검증하고 결과를 요약하여 출력 및 저장하는 작업을 수행합니다.
여기서는 `clf_lr` 모델에 대해 수행됩니다.
```

1. 교차 검증:

- `cross_validate` 함수를 사용하여 4-폴드 그룹 교차 검증을 수행합니다.
데이터는 `ColumnTransformer`를 통해 전처리된 후 로지스틱 회귀 모델로 학습됩니다.
- `groups=df_train['product_code']`를 사용하여 `product_code` 별로 그룹화하여 폴드를 나눕니다.

2. 결과 포매팅 및 출력:

- 검증 세트와 훈련 세트의 평균 ROC AUC 점수와 표준 편차를 계산하여 포매팅된 문자열로 만듭니다.
- 포매팅된 결과를 출력하고 `s_hist` 리스트에 저장합니다.
- `s_hist` 리스트를 데이터프레임으로 변환하여 각 모델의 마지막 결과를 출력합니다.

요약

이 코드는 다음과 같은 단계를 수행합니다:

1. 전처리: 특정 열을 표준화하고 다른 열을 그대로 유지합니다.
2. 모델 정의: 로지스틱 회귀 모델을 전처리와 함께 파이프라인으로 정의합니다.
3. 모델 평가: 교차 검증을 통해 모델 성능을 평가하고 결과를 출력 및 저장합니다.

이 과정을 통해 데이터 전처리와 모델 학습이 일관성 있게 수행되며, 모델의 성능을 객관적으로 평가할 수 있습니다.

```
from sklearn.linear_model import LogisticRegression

# Baseline: LR - SFS - ['loading', 'measurement_1', 'measurement_4', 'measurement_14', 'measurement_17', 'na_1']
ct = ColumnTransformer([
    ('std', StandardScaler(), ['loading', 'measurement_1', 'measurement_4', 'measurement_14', 'measurement_17']),
    ('pt', 'passthrough', ['na_1'])
])

clf_lr = make_pipeline(ct, LogisticRegression(solver='lbfgs'))

eval_model('baseline', clf_lr)
```

Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146

result**model**

baseline Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146

Step3: 모델 선택 루틴을 만듭니다.


```
In [16]: '''
코드는 `clf_lr` 모델을 학습시켜 테스트 데이터에 대한 예측을 수행하고,
예측 결과를 실제 정답과 비교하여 ROC AUC 점수를 계산하는 과정을 포함하고 있습니다.

### 모델 학습 및 예측 수행
1. 모델 학습 및 예측 함수 호출:
- `select_model` 함수를 호출하여 `clf_lr` 모델을 학습시키고 예측을 수행합니다.
- `select_model` 함수는 이전에 정의된 대로, 전체 학습 데이터를 사용하여 모델을 학습시키고,
  테스트 데이터에 대해 예측을 수행하여 예측 확률을 반환합니다.
- 반환된 예측 확률이 `prd` 변수에 저장됩니다.

2. select_model 함수 내부 동작:
- 모델 학습:
  - `clf.fit(df_train[X_all], df_train['failure'])`를 호출하여 `df_train` 데이터의
    피처 `X_all`과 타겟 `failure`를 사용하여 모델을 학습시킵니다.
- 예측 수행:
  - `clf.predict_proba(df_test[X_all])[:, 1]`를 호출하여 테스트 데이터 `df_test`에 대해
    예측 확률을 계산합니다. 이 때, 긍정 클래스(실패)에 대한 확률을 반환합니다.
- 결과 저장:
  - 예측 결과를 데이터프레임으로 만들어 'answer6.csv' 파일로 저장합니다.
- 예측 결과 반환:
  - 예측 확률 `prd`를 반환합니다.

### ROC AUC 점수 계산
3. ROC AUC 점수 계산 및 출력:
- `roc_auc_score(df_ans['failure'], prd)`를 호출하여 실제 정답 `df_ans['failure']`와
  예측 확률 `prd`를 비교하여 ROC AUC 점수를 계산합니다.
- 계산된 ROC AUC 점수를 "자가 채점"이라는 라벨과 함께 출력합니다.

### 코드의 전체 동작 요약
1. 모델 학습 및 예측 수행:
- `select_model` 함수를 통해 `clf_lr` 모델을 학습시키고, 테스트 데이터에 대해 예측 확률을 계산합니다.
- 예측 확률을 'answer6.csv' 파일로 저장하고, `prd` 변수에 반환합니다.
2. ROC AUC 점수 계산 및 출력:
- `roc_auc_score` 함수를 사용하여 실제 정답과 예측 확률을 비교하여 ROC AUC 점수를 계산합니다.
- 계산된 ROC AUC 점수를 출력합니다.

이 과정을 통해 모델의 성능을 평가하고, 예측 결과를 저장하며, ROC AUC 점수를 통해 모델의 예측 성능을 측정할 수 있습니다.
'''
```

```
prd = select_model(clf_lr)
print("자가 채점: {}".format(roc_auc_score(df_ans['failure'], prd)))
```

자가 채점: 0.5883988309352517

Step4: 모델 개선 작업을 합니다.

In [17]:

```
'''
코드는 전처리 및 모델 학습을 포함하여 새로운 파이프라인을 정의하고, 이를 사용하여 모델 성능을 교차 검증하는 과정을 수행합니다.

### 전처리와 모델 정의
1. ColumnTransformer 정의:
- PCA 적용:
  - `StandardScaler()`와 `PCA(n_components=7)`를 결합하여 파이프라인을 구성하고,
    `['measurement_{}'.format(i) for i in range(18)]` 열에 적용합니다.
    이 열들은 `measurement_0`부터 `measurement_17`까지 18개의 측정 값을 포함합니다.
  - `StandardScaler()`는 각 열의 평균을 0으로, 표준편차를 1로 맞추어 표준화합니다.
  - `PCA(n_components=7)`는 주성분 분석을 통해 7개의 주성분으로 차원을 축소합니다.
- StandardScaler 적용:
  - `StandardScaler()`를 사용하여 `['loading']` 열을 표준화합니다.
- Passthrough 적용:
  - `passthrough` 옵션을 사용하여 `['na_1']` 열은 원래의 값을 그대로 유지하여 전달합니다.

2. Logistic Regression 모델 정의:
- Pipeline 구성:
  - `ColumnTransformer`와 `LogisticRegression`을 결합하여 파이프라인을 만듭니다.
    이렇게 하면 전처리와 모델 학습이 일관된 방식으로 수행됩니다.
- Logistic Regression:
  - `LogisticRegression` 모델을 사용하며, `solver='lbfgs'`를 지정하여 최적화 알고리즘으로 `lbfgs`를 사용합니다.

### 모델 평가 함수 호출
3. 모델 평가 함수 호출:
  - `eval_model` 함수를 호출하여 'lr2'라는 이름으로 정의한 `clf_lr2` 모델을 평가합니다.
  - `eval_model` 함수는 이전에 정의한 대로 모델을 교차 검증하고 결과를 출력 및 저장합니다.

### `eval_model` 함수의 내부 동작
`eval_model` 함수는 주어진 모델을 교차 검증하고 결과를 요약하여
출력 및 저장하는 작업을 수행합니다. 여기서는 `clf_lr2` 모델에 대해 수행됩니다:

1. 교차 검증:
  - `cross_validate` 함수를 사용하여 4-폴드 그룹 교차 검증을 수행합니다.
```

데이터는 `ColumnTransformer`를 통해 전처리된 후 로지스틱 회귀 모델로 학습됩니다.

- `groups=df\_train['product\_code']`를 사용하여 `product\_code`별로 그룹화하여 폴드를 나눕니다.

2. 결과 포매팅 및 출력:

- 검증 세트와 훈련 세트의 평균 ROC AUC 점수와 표준 편차를 계산하여 포매팅된 문자열로 만듭니다.
- 포매팅된 결과를 출력하고 `s\_hist` 리스트에 저장합니다.
- `s\_hist` 리스트를 데이터프레임으로 변환하여 각 모델의 마지막 결과를 출력합니다.

코드의 전체 동작 요약

1. 전처리:

- `measurement\_0`부터 `measurement\_17`까지의 열에 대해 표준화를 수행한 후 PCA로 차원을 7개로 축소합니다.
- `loading` 열을 표준화합니다.
- `na\_1` 열은 그대로 유지합니다.

2. 모델 정의:

- 위의 전처리 과정을 포함한 로지스틱 회귀 모델을 파이프라인으로 정의합니다.

3. 모델 평가:

- `eval\_model` 함수를 통해 정의된 파이프라인 모델을 교차 검증하여 성능을 평가하고 결과를 출력 및 저장합니다.

이 과정을 통해 데이터 전처리와 모델 학습이 일관성 있게 수행되며, 모델의 성능을 객관적으로 평가할 수 있습니다.

PCA를 사용하여 차원을 축소함으로써, 데이터의 복잡성을 줄이고 모델의 효율성을 높이는 시도를 합니다.

...

```
# Lr2: LR + feature PCA(n_components = 7)
```

```
from sklearn.decomposition import PCA
```

```
ct = ColumnTransformer([
    ('std_pca', make_pipeline(StandardScaler(), PCA(n_components=7)), ['measurement_{}'.format(i) for i in range(18)]),
    ('std', StandardScaler(), ['loading']),
    ('pt', 'passthrough', ['na_1'])
])
```

```
clf_lr2 = make_pipeline(ct, LogisticRegression(solver='lbfgs'))
```

```
eval_model('lr2', clf_lr2)
```

```
Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
```

result	
model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103

In [38]:

```
'''
코드는 전처리와 모델 학습을 결합하여 새로운 파이프라인을 정의하고, 이를 사용하여 모델 성능을 교차 검증하는 과정을 수행합니다.
특히, 'loading' 열에 로그 변환을 적용한 후 표준화를 수행하는 특징이 있습니다.
```

```
# 전처리와 모델 정의
```

```
1. ColumnTransformer 정의:
```

- StandardScaler 적용:
 - `StandardScaler()`를 사용하여 `[ 'measurement\_1', 'measurement\_4', 'measurement\_14', 'measurement\_17' ]` 열을 표준화합니다. 표준화는 각 열의 평균을 0으로, 표준편차를 1로 맞추는 작업입니다.
- 로그 변환 후 표준화 적용:
 - `make\_pipeline(FunctionTransformer(func=np.log, validate=False), StandardScaler())`를 사용하여 `[ 'loading' ]` 열에 로그 변환을 적용한 후 표준화를 수행합니다.
 - `FunctionTransformer(func=np.log, validate=False)`는 `np.log` 함수를 사용하여 로그 변환을 수행합니다. `validate=False`는 입력 데이터의 유효성을 검사하지 않음을 의미합니다.
 - 변환된 값에 대해 `StandardScaler()`를 사용하여 표준화를 수행합니다.
- Passthrough 적용:
 - `passthrough` 옵션을 사용하여 `[ 'na\_1' ]` 열은 원래의 값을 그대로 유지하여 전달합니다.

```
2. Logistic Regression 모델 정의:
```

- Pipeline 구성:
 - `ColumnTransformer`와 `LogisticRegression`을 결합하여 파이프라인을 만듭니다. 이렇게 하면 전처리와 모델 학습이 일관된 방식으로 수행됩니다.
- Logistic Regression:
 - `LogisticRegression` 모델을 사용하며, `solver='lbfgs'`를 지정하여 최적화 알고리즘으로 `lbfgs`를 사용합니다.

```
# 모델 평가 함수 호출
```

```
3. 모델 평가 함수 호출:
```

- `eval\_model` 함수를 호출하여 'lr3'라는 이름으로 정의한 `clf\_lr3` 모델을 평가합니다.
- `eval\_model` 함수는 이전에 정의한 대로 모델을 교차 검증하고 결과를 출력 및 저장합니다.

```
# `eval_model` 함수의 내부 동작
```

```
`eval_model` 함수는 주어진 모델을 교차 검증하고 결과를 요약하여 출력 및 저장하는 작업을 수행합니다.
```

```
여기서는 `clf_lr3` 모델에 대해 수행됩니다:
```

```
1. 교차 검증:
```

- ``cross_validate`` 함수를 사용하여 4-폴드 그룹 교차 검증을 수행합니다.
데이터는 ``ColumnTransformer``를 통해 전처리된 후 로지스틱 회귀 모델로 학습됩니다.
- ``groups=df_train['product_code']``를 사용하여 ``product_code`` 별로 그룹화하여 폴드를 나눕니다.

2. 결과 포매팅 및 출력:

- 검증 세트와 훈련 세트의 평균 ROC AUC 점수와 표준 편차를 계산하여 포매팅된 문자열로 만듭니다.
- 포매팅된 결과를 출력하고 ``s_hist`` 리스트에 저장합니다.
- ``s_hist`` 리스트를 데이터프레임으로 변환하여 각 모델의 마지막 결과를 출력합니다.

코드의 전체 동작 요약

1. 전처리:

- ``measurement_1``, ``measurement_4``, ``measurement_14``, ``measurement_17`` 열을 표준화합니다.
- ``loading`` 열에 로그 변환을 적용한 후 표준화를 수행합니다.
- ``na_1`` 열은 그대로 유지합니다.

2. 모델 정의:

- 위의 전처리 과정을 포함한 로지스틱 회귀 모델을 파이프라인으로 정의합니다.

3. 모델 평가:

- ``eval_model`` 함수를 통해 정의된 파이프라인 모델을 교차 검증하여 성능을 평가하고 결과를 출력 및 저장합니다.

이 과정을 통해 데이터 전처리와 모델 학습이 일관성 있게 수행되며, 모델의 성능을 객관적으로 평가할 수 있습니다.
특히, ``loading`` 열에 로그 변환을 적용하여 값의 분포를 조정함으로써 모델 성능을 개선할 수 있습니다.

```
# lr3: Baseline + np.log (Loading)
```

```
from sklearn.preprocessing import FunctionTransformer
```

```
ct = ColumnTransformer([
    ('std', StandardScaler(), ['measurement_1', 'measurement_4', 'measurement_14', 'measurement_17']),
    ('log_std', make_pipeline(FunctionTransformer(func=np.log, validate=False),
                             StandardScaler()), ['loading']),
    ('pt', 'passthrough', ['na_1'])
])
```

```
clf_lr3 = make_pipeline(ct, LogisticRegression(solver='lbfgs'))
```

```
eval_model('lr3', clf_lr3)
```

```
Valid: 0.58908±0.00448, V.Train: 0.59099±0.00156
```

result

model

baseline Valid: 0.58920±0.00444, V.Train: 0.59119±0.00150

lr2 Valid: 0.58834±0.00384, V.Train: 0.59087±0.00114

lr3 Valid: 0.58908±0.00448, V.Train: 0.59099±0.00156

In [19]:

```
'''
주어진 코드는 `LinearDiscriminantAnalysis (LDA)` 모델을 사용하여 데이터의 분류 작업을 수행하는
과정에서 특정 속성(특징)을 선택하고, 데이터를 표준화한 후 평가하는 작업을 수행합니다.

# 단계별 설명
1. 속성 선택 및 전처리
    - `ColumnTransformer`는 여러 열에 대해 다양한 전처리를 적용할 수 있는 도구입니다.
    - 두 가지 전처리 전략이 사용됩니다:
        - `StandardScaler`를 사용하여 선택된 속성들을 표준화합니다.
        - `passthrough` 옵션을 사용하여 특정 속성은 전처리 없이 그대로 통과시킵니다.
    - 표준화되는 속성들: `loading`, `measurement_1`, `measurement_4`, `measurement_14`, `measurement_17`
    - 그대로 통과하는 속성: `na_1`

2. 파이프라인 생성
    - `make_pipeline` 함수를 사용하여 전처리 과정과 `LinearDiscriminantAnalysis (LDA)` 모델을 하나의 파이프라인으로 결합합니다.
    - 데이터가 파이프라인을 통과할 때, 먼저 `ColumnTransformer`를 통해 전처리가 수행되고, 그 다음 `LDA` 모델이 적용됩니다.

3. 모델 평가
    - `eval_model` 함수는 'lda'라는 이름과 함께 `clf_lda` 모델을 입력으로 받아 모델을 평가합니다.
    - `eval_model` 함수가 어떤 방식으로 평가를 수행하는지는 주어진 코드에서는 알 수 없지만,
      일반적으로 교차 검증, 정확도 계산, 혼동 행렬 작성 등의 작업을 수행할 것입니다.

# 전체적인 흐름
- 선택된 속성들을 표준화하고 특정 속성은 그대로 통과시키는 전처리 과정을 정의합니다.
- 전처리 과정과 `LDA` 모델을 하나의 파이프라인으로 결합합니다.
- 결합된 파이프라인을 평가하여 모델의 성능을 확인합니다.

이 과정을 통해 모델에 사용되는 데이터를 적절히 전처리하고, 선택된 속성들에 대해 LDA 모델을 적용하여 분류 작업을 수행할 수 있습니다.
'''

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# LDA
# 속성선택도 적용해봅니다:
clf_lda = make_pipeline(
    ColumnTransformer([
        #('std', StandardScaler(), ['measurement_{}'.format(i) for i in range(18)] + ['loading']),
        ('std', StandardScaler(), ['loading', 'measurement_1', 'measurement_4', 'measurement_14', 'measurement_17']),
        ('pt', 'passthrough', ['na_1'])
    ]),
    LinearDiscriminantAnalysis()
)

eval_model('lda', clf_lda)
```

Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148

result

model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lda	Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
lr3	Valid: 0.58935±0.00389, V.Train: 0.59181±0.00145
xgb	Valid: 0.58347±0.00335, V.Train: 0.62258±0.00110

In [20]:

```
'''
주어진 코드는 `RandomForestClassifier`를 사용하여 데이터를 분류하는 작업을 수행하는 과정에서,
열을 그대로 통과시키는 전처리 단계를 거친 후 모델을 평가하는 작업을 수행합니다.

# 단계별 설명

1. 열 변환기 정의
- `ColumnTransformer`는 여러 열에 대해 다양한 전처리를 적용할 수 있는 도구입니다.
- 여기서는 `passthrough` 전략을 사용하여 지정된 열들을 전처리 없이 그대로 통과시키도록 합니다.
- 그대로 통과시키는 열들: `loading`, `na_1`, `na_2` 및 `measurement_0`부터 `measurement_17`까지의 열들

2. 파이프라인 생성
- `make_pipeline` 함수를 사용하여 전처리 과정(`ct`)과 `RandomForestClassifier` 모델을 하나의 파이프라인으로 결합합니다.
- `RandomForestClassifier`의 하이퍼파라미터 설정:
    - `n_estimators=100`: 100개의 의사결정 나무를 생성합니다.
'''
```

- ``max_depth=6``: 나무의 최대 깊이는 6으로 제한합니다.
- ``min_samples_split=512``: 내부 노드를 분할하는 데 필요한 최소 샘플 수는 512입니다.
- ``random_state=123``: 랜덤 시드를 123으로 설정하여 결과의 재현성을 보장합니다.
- ``max_features=0.75``: 각 나무에서 사용할 특징 수는 전체 특징 수의 75%입니다.

3. 모델 평가

- ``eval_model`` 함수는 'rf'라는 이름과 함께 ``clf_rf`` 모델을 입력으로 받아 모델을 평가합니다.
- ``eval_model`` 함수가 어떤 방식으로 평가를 수행하는지는 주어진 코드에서는 알 수 없지만, 일반적으로 교차 검증, 정확도 계산, 혼동 행렬 작성 등의 작업을 수행할 것입니다.

전체적인 흐름

- 특정 열들을 그대로 통과시키는 전처리 과정을 정의합니다.
- 전처리 과정과 ``RandomForestClassifier`` 모델을 하나의 파이프라인으로 결합합니다.
- 결합된 파이프라인을 평가하여 모델의 성능을 확인합니다.

이 과정을 통해 모델에 사용되는 데이터를 전처리 없이 그대로 사용하고, ``RandomForestClassifier``를 적용하여 분류 작업을 수행할 수 있습니다.

RF: RandomForestClassifier: {'max_depth': 7, 'min_samples_split': 512, 'n_estimators': 15} 튜닝

```
from sklearn.ensemble import RandomForestClassifier
```

```
ct = ColumnTransformer([
    ('pt', 'passthrough', ['loading', 'na_1', 'na_2'] + ['measurement_{}'.format(i) for i in range(18)])
])
```

```
clf_rf = make_pipeline(ct, RandomForestClassifier(
    n_estimators=100, max_depth=6, min_samples_split=512, random_state=123, max_features=0.75
))
```

```
eval_model('rf', clf_rf)
```

Valid: 0.58163±0.00300, V.Train: 0.63956±0.00229

result	
model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lda	Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
lr3	Valid: 0.58935±0.00389, V.Train: 0.59181±0.00145
rf	Valid: 0.58163±0.00300, V.Train: 0.63956±0.00229
xgb	Valid: 0.58347±0.00335, V.Train: 0.62258±0.00110

In [21]:

```
'''
주어진 코드는 `RandomForestClassifier`를 사용하여 데이터를 분류하는 작업을 수행하는 과정에서,
일부 열에 대해 `LinearDiscriminantAnalysis (LDA)` 전처리를 적용한 후 모델을 평가하는 작업을 수행합니다.

# 단계별 설명
1. 라이브러리 임포트 및 함수 정의 (생략된 부분)
    - `make_pipeline`, `ColumnTransformer`, `StandardScaler`, `LinearDiscriminantAnalysis`,
      `RandomForestClassifier` 등 필요한 라이브러리와 함수들이 임포트되어 있어야 합니다.
    - `eval_model`이라는 함수가 정의되어 있어야 하며, 이 함수는 모델을 평가하는 역할을 합니다.

2. 열 변환기 정의
    - `ColumnTransformer`는 여러 열에 대해 다양한 전처리를 적용할 수 있는 도구입니다.
    - 두 가지 변환 전략이 사용됩니다:
        - 첫 번째 변환 ('lda'):
            - 선택된 열들에 대해 먼저 `StandardScaler`를 적용하여 데이터를 표준화한 후, `LinearDiscriminantAnalysis (LDA)`를 적용함
            - 선택된 열들: `measurement_0`부터 `measurement_17`까지
        - 두 번째 변환 ('pt'):
            - 특정 열들은 전처리 없이 그대로 통과시킵니다.
            - 그대로 통과하는 열들: `loading`, `na_1`, `na_2`

3. 파이프라인 생성
    - `make_pipeline` 함수를 사용하여 전처리 과정(`ct`)과 `RandomForestClassifier` 모델을 하나의 파이프라인으로 결합합니다.
    - `RandomForestClassifier`의 하이퍼파라미터 설정:
        - `n_estimators=100`: 100개의 의사결정 나무를 생성합니다.
        - `max_depth=6`: 나무의 최대 깊이는 6으로 제한합니다.
        - `min_samples_split=512`: 내부 노드를 분할하는 데 필요한 최소 샘플 수는 512입니다.
        - `random_state=123`: 랜덤 시드를 123으로 설정하여 결과의 재현성을 보장합니다.
'''
```

- ``max_features=0.75``: 각 나무에서 사용할 특징 수는 전체 특징 수의 75%입니다.

4. 모델 평가

- `eval_model`: 주어진 모델을 평가하는 함수입니다.
- `model_name`: 모델의 이름을 지정합니다. 여기서는 ``rf2``로 지정되어 있습니다.
- `clf`: 평가할 분류기입니다. 여기서는 ``clf_rf2``가 전달됩니다.

``eval_model`` 함수 내부 동작

1. 교차 검증 실행:

- `cross_validate` 함수를 사용하여 교차 검증을 수행합니다.
- 입력 데이터는 ``df_train[X_all]``, 목표 변수는 ``df_train['failure']``입니다.
- 그룹화 기준은 ``df_train['product_code']``입니다.
- 평가 지표로는 ROC AUC 스코어를 사용합니다.

2. 결과 출력:

- 교차 검증 결과에서 테스트와 훈련 점수의 평균 및 표준 편차를 계산합니다.
- 결과를 형식화하여 출력하고, ``s_hist`` 리스트에 저장합니다.

3. 결과 표시:

- ``pd.DataFrame(s_hist).groupby('model').last()``를 통해 모델별 마지막 결과를 표시합니다.

이렇게 ``eval_model`` 함수는 모델을 평가하고, 그 결과를 출력하며 기록하는 역할을 합니다.

전체적인 흐름

- 일부 열들에 대해 표준화와 LDA 전처리를 적용하고, 다른 열들은 전처리 없이 그대로 통과시키는 전처리 과정을 정의합니다.
- 전처리 과정과 ``RandomForestClassifier`` 모델을 하나의 파이프라인으로 결합합니다.
- 결합된 파이프라인을 평가하여 모델의 성능을 확인합니다.

이 과정을 통해 모델에 사용되는 데이터를 적절히 전처리하고, ``RandomForestClassifier``를 적용하여 분류 작업을 수행할 수 있습니다.

LDA를 사용하여 변환된 속성들은 데이터의 차원을 줄이고,

랜덤 포레스트 모델은 이러한 변환된 데이터와 원본 데이터를 함께 사용하여 분류 작업을 수행합니다.

...

RF: *RandomForestClassifier + LinearDiscriminantAnalysis*

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
```

```
ct = ColumnTransformer([
    ('lda', make_pipeline(StandardScaler(), LinearDiscriminantAnalysis()),
     ['measurement_{}'.format(i) for i in range(18)]),
    # + ['measurement_{}'.format(i) for i in range(18)]
    ('pt', 'passthrough', ['loading', 'na_1', 'na_2'])
```

```

])

clf_rf2 = make_pipeline(ct, RandomForestClassifier(
    n_estimators=100, max_depth=6, min_samples_split=512,
    random_state=123, max_features=0.75
))

eval_model('rf2', clf_rf2)

```

Valid: 0.58567±0.00221, V.Train: 0.62854±0.00269

result

model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lda	Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
lr3	Valid: 0.58935±0.00389, V.Train: 0.59181±0.00145
rf	Valid: 0.58163±0.00300, V.Train: 0.63956±0.00229
rf2	Valid: 0.58567±0.00221, V.Train: 0.62854±0.00269
xgb	Valid: 0.58347±0.00335, V.Train: 0.62258±0.00110

In [25]: '''

이 코드는 데이터 전처리와 모델 학습을 결합하여 평가하는 파이프라인을 생성합니다.

1. `ColumnTransformer` 설정

`ColumnTransformer`는 서로 다른 전처리 과정을 특정 열에 적용할 수 있게 해줍니다.

이 예제에서는 `passthrough` 옵션을 사용하여 선택된 열을 그대로 전달합니다.

- `'pt'`: 이 변환기의 이름입니다.

- `'passthrough'`: 이 옵션은 지정된 열을 전혀 변환하지 않고 그대로 전달합니다.

- `['loading', 'na\_1', 'na\_2'] + ['measurement\_{}'.format(i) for i in range(18)]`:

이 리스트는 변환기에 전달할 열들의 이름을 포함합니다.

- `'loading'`, `'na\_1'`, `'na\_2'`는 직접 명시된 열 이름입니다.

- `['measurement\_{}'.format(i) for i in range(18)]`는 `measurement\_0`에서 `measurement\_17`까지의 열 이름을 생성합니다.

2. `XGBClassifier` 설정

XGBoost 분류기를 설정합니다.

- `n\_estimators=300`: 300개의 결정 트리를 사용합니다.

- ``learning_rate=0.007``: 학습 속도를 **0.007**로 설정합니다.
- ``colsample_bytree=0.9``: 각 트리를 만들 때 무작위로 선택할 열의 비율을 **0.9**로 설정합니다.
- ``subsample=0.75``: 각 트리를 만들 때 무작위로 선택할 데이터 포인트의 비율을 **0.75**로 설정합니다.
- ``max_depth=2``: 각 트리의 최대 깊이를 **2**로 설정합니다.

3. 파이프라인 생성

전처리와 모델을 결합한 파이프라인을 만듭니다.

- ``make_pipeline(ct, clf_xgb)``: ``ColumnTransformer``와 ``XGBClassifier``를 순차적으로 실행하는 파이프라인을 만듭니다.
 - 첫 번째 단계에서는 ``ct``가 적용되어 선택된 열들을 그대로 전달합니다.
 - 두 번째 단계에서는 ``XGBClassifier``가 적용되어 분류 작업을 수행합니다.

4. 모델 평가

``eval_model`` 함수를 사용하여 모델을 평가합니다.

- ``eval_model('xgb', clf_xgb)``: ``'xgb'``라는 이름으로 파이프라인 ``clf_xgb``를 평가합니다.
 - ``eval_model`` 함수는 주어진 파이프라인을 평가하는 사용자 정의 함수일 가능성이 높습니다. 데이터 분할, 모델 학습, 예측 및 성능 평가를 수행할 수 있습니다.

요약

이 코드는 특정 열을 그대로 전달하고 XGBoost 분류기를 사용하여 예측하는 파이프라인을 설정하고, 이를 평가하는 과정입니다. 이 과정은 데이터 전처리와 모델 학습을 하나의 파이프라인으로 결합하여 효율적으로 처리할 수 있게 합니다.

XGB: XGBoost

```
import xgboost as xgb
ct = ColumnTransformer([
    ('pt', 'passthrough', ['loading', 'na_1', 'na_2'] + ['measurement_{}'.format(i) \
                                                         for i in range(18)])
])

clf_xgb = xgb.XGBClassifier(n_estimators=300, learning_rate=0.007,
                           colsample_bytree=0.9, subsample=0.75, max_depth=2)
clf_xgb = make_pipeline(ct, clf_xgb)

eval_model('xgb', clf_xgb)
```

Valid: 0.58616±0.00399, V.Train: 0.60347±0.00080

result	
model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lda	Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
lr3	Valid: 0.58935±0.00389, V.Train: 0.59181±0.00145
rf	Valid: 0.58163±0.00300, V.Train: 0.63956±0.00229
rf2	Valid: 0.58567±0.00221, V.Train: 0.62854±0.00269
vt	Valid: 0.58973±0.00347, V.Train: 0.61029±0.00148
xgb	Valid: 0.58616±0.00399, V.Train: 0.60347±0.00080

In [28]:

```
'''
코드는 데이터 전처리와 모델 학습을 결합하여 평가하는 파이프라인을 생성합니다.

# 1. `ColumnTransformer` 설정
`ColumnTransformer`는 서로 다른 전처리 과정을 특정 열에 적용할 수 있게 해줍니다.
이 예제에서는 `passthrough` 옵션을 사용하여 선택된 열을 그대로 전달합니다.
- `'pt'`: 이 변환기의 이름입니다.
- `'passthrough'`: 이 옵션은 지정된 열을 전혀 변환하지 않고 그대로 전달합니다.
- `['loading', 'na_1', 'na_2'] + ['measurement_{}'.format(i) for i in range(18)]:`
  이 리스트는 변환기에 전달할 열들의 이름을 포함합니다.
- `'loading', 'na_1', 'na_2'`는 직접 명시된 열 이름입니다.
- `['measurement_{}'.format(i) for i in range(18)]`는 `measurement_0`에서 `measurement_17`까지의 열 이름을 생성합니다.

# 2. `XGBClassifier` 설정
XGBoost 분류기를 설정합니다.
- `n_estimators=300`: 300개의 결정 트리를 사용합니다.
- `learning_rate=0.007`: 학습 속도를 0.007로 설정합니다.
- `colsample_bytree=0.9`: 각 트리를 만들 때 무작위로 선택할 열의 비율을 0.9로 설정합니다.
- `subsample=0.75`: 각 트리를 만들 때 무작위로 선택할 데이터 포인트의 비율을 0.75로 설정합니다.
- `max_depth=2`: 각 트리의 최대 깊이를 2로 설정합니다.

# 3. 파이프라인 생성
전처리와 모델을 결합한 파이프라인을 만듭니다.
- `make_pipeline(ct, clf_xgb)`: `ColumnTransformer`와 `XGBClassifier`를 순차적으로 실행하는 파이프라인을 만듭니다.
```

- 첫 번째 단계에서는 `ct`가 적용되어 선택된 열들을 그대로 전달합니다.
- 두 번째 단계에서는 `XGBClassifier`가 적용되어 분류 작업을 수행합니다.

4. 모델 평가

`eval\_model` 함수를 사용하여 모델을 평가합니다.

- `eval\_model('xgb', clf\_xgb)`: `xgb`라는 이름으로 파이프라인 `clf\_xgb`를 평가합니다.
- `eval\_model` 함수는 주어진 파이프라인을 평가하는 사용자 정의 함수입니다.

요약

이 코드는 특정 열을 그대로 전달하고 XGBoost 분류기를 사용하여 예측하는 파이프라인을 설정하고, 이를 평가하는 과정입니다. 이 과정은 데이터 전처리와 모델 학습을 하나의 파이프라인으로 결합하여 효율적으로 처리할 수 있게 합니다.

```
'''
```

Voting을 합니다.

```
from sklearn.ensemble import VotingClassifier
```

```
clf_vt = VotingClassifier([
    ('baseline', clf_lr),
    ('lr2', clf_lr2),
    ('lr3', clf_lr3),
    ('rf', clf_rf),
    ('rf2', clf_rf2),
    ('lda', clf_lda),
    ('xgb', clf_xgb),
], voting='soft')
eval_model('vt', clf_vt)
```

Valid: 0.58999±0.00346, V.Train: 0.60794±0.00154

result	
model	
baseline	Valid: 0.58937±0.00379, V.Train: 0.59191±0.00146
lda	Valid: 0.58963±0.00396, V.Train: 0.59196±0.00148
lr2	Valid: 0.58857±0.00294, V.Train: 0.59163±0.00103
lr3	Valid: 0.58935±0.00389, V.Train: 0.59181±0.00145
rf	Valid: 0.58163±0.00300, V.Train: 0.63956±0.00229
rf2	Valid: 0.58567±0.00221, V.Train: 0.62854±0.00269
vt	Valid: 0.58999±0.00346, V.Train: 0.60794±0.00154
xgb	Valid: 0.58616±0.00399, V.Train: 0.60347±0.00080

```
In [29]: prd = select_model(clf_vt)
print("자가 채점: {}".format(roc_auc_score(df_ans['failure'], prd)))
```

자가 채점: 0.5920674460431654

```
In [30]: # ChatGPT 3.5 적용
...
```

코드는 주어진 모델 리스트를 기반으로 각 모델을 학습시키고,
테스트 데이터에 대한 예측 확률을 뽑아서 데이터프레임에 저장하는 과정입니다.

1. 모델 리스트 설정

`model\_list`는 다양한 모델과 해당 모델의 이름으로 구성된 리스트입니다.
각 튜플의 첫 번째 요소는 모델의 이름이고, 두 번째 요소는 해당 모델입니다.

2. 테스트 데이터에 대한 예측 확률 계산

주어진 테스트 데이터(`df\_test`)에 대해 각 모델을 학습시키고, 예측 확률을 계산합니다.

- `pd.Series(...)`으로 각 모델에 대한 예측 확률을 시리즈 형태로 변환합니다.
- `clf.fit(df\_train[X\_all], df\_train['failure'])`를 사용하여 모델을 학습시킵니다.
이때 `df\_train[X\_all]`은 모델에 사용될 특성들을 나타냅니다.
- `predict\_proba(df\_test[X\_all])[:, 1]`를 사용하여 테스트 데이터에 대한 예측 확률 중 양성 클래스(1)에 대한 확률만을 선택합니다.
- `index=df\_test.index`를 통해 예측 결과의 인덱스를 `df\_test`의 인덱스로 설정합니다.
- `name=model\_name`을 통해 시리즈의 이름을 각 모델의 이름으로 설정합니다.

3. 결과 데이터프레임 생성

`pd.concat`을 사용하여 각 모델의 예측 확률을 하나의 데이터프레임으로 결합합니다.
 - `axis=1`은 열 방향으로 데이터프레임을 결합한다는 것을 의미합니다.

이렇게 하면 `df\_prds`에는 각 모델의 이름을 열 이름으로,
 각 데이터 포인트에 대한 예측 확률을 값으로 갖는 데이터프레임이 생성됩니다.
 ...

...

코드는 주어진 모델 리스트를 기반으로 각 모델을 학습시키고, 테스트 데이터에 대한
 예측 확률을 계산하여 하나의 데이터프레임으로 결합하는 과정을 수행합니다.

1. 모델 리스트 설정

`model\_list`는 다양한 모델과 해당 모델의 이름으로 구성된 리스트입니다.
 각 튜플의 첫 번째 요소는 모델의 이름이고, 두 번째 요소는 해당 모델 객체입니다.
 각 모델은 다른 전처리 방법이나 하이퍼파라미터 조정을 통해 설정되었습니다.

2. 테스트 데이터에 대한 예측 확률 계산

각 모델을 학습시키고, 테스트 데이터에 대한 예측 확률을 계산하여 데이터프레임에 저장합니다.
 이 코드는 리스트 컴프리헨션을 사용하여 각 모델의 예측 확률을 계산하고,
 이를 시리즈 형태로 변환한 후, 데이터프레임으로 결합합니다.

내부 동작 설명

- `clf.fit(df\_train[X\_all], df\_train['failure'])`: 각 모델을 훈련 데이터(`df\_train`)로 학습시킵니다. `X\_all`은 모든 피처를 나타내며, `failure`는 타겟 변수입니다.
- `.predict\_proba(df\_test[X\_all])[:, 1]`:
 학습된 모델을 사용하여 테스트 데이터(`df\_test`)에 대한 예측 확률을 계산합니다.
 `predict\_proba`는 각 클래스에 대한 확률을 반환하며, `[:, 1]`은 양성 클래스(1)에 대한 확률을 선택합니다.
- `pd.Series(..., index=df\_test.index, name=model\_name)`: 예측 확률을 시리즈로 변환합니다.
 - `index=df\_test.index`: 테스트 데이터의 인덱스를 유지합니다.
 - `name=model\_name`: 시리즈의 이름을 모델의 이름으로 설정합니다.

리스트 컴프리헨션

리스트 컴프리헨션을 사용하여 위의 작업을 `model\_list`에 있는 모든 모델에 대해 반복합니다.

데이터프레임 결합

`pd.concat(..., axis=1)`을 사용하여 각 모델의 예측 확률 시리즈를 열 방향으로 결합하여 하나의 데이터프레임을 만듭니다.
 결과적으로 `df\_prds` 데이터프레임에는 각 모델의 예측 확률이 열로 추가됩니다.

요약

이 코드는 다음과 같은 작업을 수행합니다:

1. 여러 모델을 `df\_train` 데이터로 학습시킵니다.

2. 학습된 모델을 사용하여 `df\_test` 데이터에 대한 예측 확률을 계산합니다.
3. 각 모델의 예측 확률을 시리즈로 변환하고, 이를 열 방향으로 결합하여 하나의 데이터프레임 `df\_prds`를 만듭니다.

결과적으로, `df\_prds` 데이터프레임에는 각 모델의 이름을 열 이름으로, 각 데이터 포인트에 대한 예측 확률을 값으로 가지는 열들이 포함됩니다. 이 데이터프레임은 나중에 예측 성능 평가나 앙상블 모델링에 사용할 수 있습니다.

```

...

# Voting 을 직접해 봅니다.
model_list = [
    ('baseline', clf_lr), # LR + SFS
    ('lr2', clf_lr2), # LR.2: LR + feature PCA
    ('lr3', clf_lr3), # Basline + np.Log (Loading)
    ('lda', clf_lda), # LDA
    ('rf', clf_rf), # RF 튜닝
    ('rf2', clf_rf2), # RF + LDA
    ('xgb', clf_xgb), # XGB
]

# 기반 모델을 df_train으로 학습시킨 후, 바로 df_test에 대한 예측 확률을 뽑습니다.
df_prds = pd.concat([
    pd.Series(
        clf.fit(df_train[X_all], df_train['failure'])\
            .predict_proba(df_test[X_all])[ :, 1],
        index=df_test.index, name=model_name
    )
    for model_name, clf in model_list
], axis=1)

df_prds.head()

```

Out[30]:

	baseline	lr2	lr3	lda	rf	rf2	xgb
id							
16115	0.145254	0.157725	0.132483	0.140965	0.131637	0.139301	0.188771
16116	0.178405	0.170402	0.175762	0.174935	0.169701	0.182337	0.214275
16117	0.215412	0.215538	0.222290	0.214288	0.205623	0.206804	0.238981
16118	0.263162	0.265910	0.266052	0.267748	0.275081	0.233135	0.300727
16119	0.215025	0.208951	0.220705	0.213243	0.203600	0.220509	0.237953

In [31]:

```
'''
코드는 자가 채점을 수행하여 각 모델의 예측 확률의 평균에 대한 ROC AUC 점수를 출력합니다.

# 1. ROC AUC 점수 계산
`roc_auc_score` 함수를 사용하여 ROC AUC 점수를 계산합니다.
- `df_ans['failure']`: 실제 타겟 변수(레이블)입니다.
  이것은 예측된 값과 비교하여 모델의 성능을 측정하는 데 사용됩니다.
- `df_prds.mean(axis=1)`: 각 행의 예측 확률의 평균을 계산합니다.
  `df_prds`는 각 모델의 예측 확률을 열로 가지고 있으므로, `axis=1`을 사용하여 각 데이터 포인트에 대한 평균을 구합니다.

# 2. 출력
계산된 ROC AUC 점수를 출력합니다.
- `"자가 채점: "`: 출력문의 시작 부분입니다.
- `roc_auc_score(df_ans['failure'], df_prds.mean(axis=1))`: 계산된 ROC AUC 점수가 여기에 채워집니다.
- `format()` 함수를 사용하여 출력 양식을 지정합니다.

# 요약
코드는 각 모델의 예측 확률의 평균에 대한 ROC AUC 점수를 계산하여 출력합니다.
ROC AUC 점수는 모델의 예측 성능을 평가하는 데 널리 사용되는 지표 중 하나이며,
이 점수는 0에서 1 사이의 값을 가집니다. 1에 가까울수록 더 좋은 성능을 나타냅니다.
'''

print("자가 채점: {}".format(roc_auc_score(df_ans['failure'], df_prds.mean(axis=1))))
```

자가 채점: 0.59206654676259

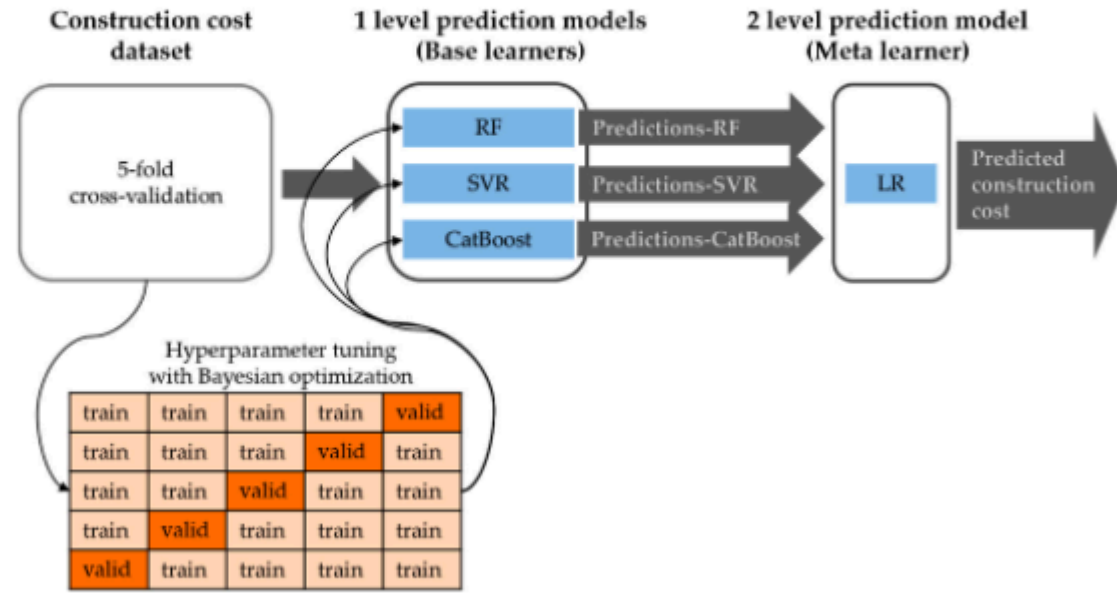
In [19]:

Image('3.png')

Out[19]: **Stacking** ¶

다른 앙상블 기법인 Stacking을 보여드립니다.

참고용입니다. 지금까지 했던 데이터 처리와 머신러닝 기법을 복습해보기 위해 준비했습니다.



※ 이미지 출처: <https://www.mdpi.com/2076-3417/12/19/9729>

In [32]:

```
'''
코드는 메타 모델을 위한 학습 셋을 만들기 위해 교차 검증을 사용하여 예측 확률을 계산하고,
이를 데이터프레임으로 결합하는 과정을 수행합니다.

# 1. 모델 리스트 설정
`model_list`는 다양한 모델과 해당 모델의 이름으로 구성된 리스트입니다.
각 튜플의 첫 번째 요소는 모델의 이름이고, 두 번째 요소는 해당 모델 객체입니다.
각 모델은 다른 전처리 방법이나 하이퍼파라미터 조정을 통해 설정되었습니다.

# 2. 메타 모델을 위한 학습 셋 생성
교차 검증을 사용하여 예측 확률을 계산하고, 이를 데이터프레임으로 결합합니다.
## 내부 동작 설명
- `cross_val_predict`: 교차 검증을 수행하여 각 분할(fold)에서 예측을 수행합니다.
- `clf`: 학습할 모델.
```

```

- `df_train[X_all]`: 입력 특성 데이터셋.
- `df_train['failure']`: 타겟 변수.
- `groups=df_train['product_code']`: 그룹 정보를 사용하여 그룹별로 교차 검증을 수행합니다.
  이는 동일한 제품 코드에 속하는 샘플이 동일한 폴드에 속하게 합니다.
- `cv=gcv`: 교차 검증 분할 전략. `gcv`는 GroupKFold 등의 그룹 기반 교차 검증 객체일 가능성이 높습니다.
- `method='predict_proba'`: 예측 확률을 계산합니다.
- `cross_val_predict(..., method='predict_proba')[:, 1]`: 각 폴드에서의 예측 확률 중 양성 클래스(1)에 대한 확률을 선택합니다.
- `pd.Series(..., index=df_train.index, name=model_name)`: 예측 확률을 시리즈로 변환합니다.
- `index=df_train.index`: 학습 데이터의 인덱스를 유지합니다.
- `name=model_name`: 시리즈의 이름을 모델의 이름으로 설정합니다.

## 리스트 컴프리헨션
리스트 컴프리헨션을 사용하여 위의 작업을 `model_list`에 있는 모든 모델에 대해 반복합니다.

## 데이터프레임 결합
`pd.concat(..., axis=1)`을 사용하여 각 모델의 예측 확률 시리즈를 열 방향으로 결합하여 하나의 데이터프레임을 만듭니다.
결과적으로 `df_cv` 데이터프레임에는 각 모델의 예측 확률이 열로 추가됩니다.

# 요약
이 코드는 다음과 같은 작업을 수행합니다:
1. 여러 모델을 사용하여 교차 검증을 수행합니다.
2. 각 폴드에서의 예측 확률을 계산합니다.
3. 예측 확률을 시리즈로 변환하고, 이를 열 방향으로 결합하여 하나의 데이터프레임 `df_cv`를 만듭니다.

결과적으로, `df_cv` 데이터프레임에는 각 모델의 이름을 열 이름으로, 각 데이터 포인트에 대한 예측 확률을 값으로 가지는 열들이 포함됩니다.
이 데이터프레임은 메타 모델(양상불 모델의 두 번째 단계) 학습에 사용할 수 있습니다.
'''

# 1단계: Cross Validation 예측결과를 모아 메타 모델 학습을 위한 데이터셋을 만듭니다.

# 메타 모델이 접하게되는 데이터는 1차 모델의 예측 결과 인데,
# 일반화 성능을 높이려면 이 데이터의 분포가
# 학습하지 않았던 데이터의 분포와 비슷해야 합니다.
# 학습에 사용되지 않았던 겹외셋의 예측 결과 사용하여,
# 이렇게 데이터의 분포를 실제 예측시의 분포와 최대한 유사하게 해줍니다.
from sklearn.model_selection import cross_val_predict

model_list = [
    ('baseline', clf_lr), # LR + SFS
    ('lr2', clf_lr2), # LR.2: LR + feature PCA
    ('lr3', clf_lr3), # Basline + np.Log (Loading)
    ('lda', clf_lda), # LDA

```

```

('rf', clf_rf), # RF 튜닝
('rf2', clf_rf2), # RF + LDA
('xgb', clf_xgb), # XGB
]

# 메타모델을 위한 학습셋을 만듭니다.
# cross_val_predict는 교차검증의 예측 결과를 입력 변수의 순서에 맞게 제공합니다.
df_cv = pd.concat([
    pd.Series(
        cross_val_predict(
            clf, df_train[X_all], df_train['failure'],
            groups=df_train['product_code'],
            cv=gcv, method='predict_proba'
        )[:, 1],
        index=df_train.index, name=model_name
    )
    for model_name, clf in model_list
], axis=1)

df_cv.head()

```

Out[32]:

	baseline	lr2	lr3	lda	rf	rf2	xgb
id							
0	0.152662	0.163569	0.141975	0.148474	0.127604	0.144275	0.191116
1	0.158010	0.160426	0.150114	0.154085	0.150513	0.142946	0.188165
2	0.166727	0.176558	0.156785	0.162389	0.139127	0.174218	0.186989
3	0.195070	0.198561	0.194955	0.191887	0.194668	0.197157	0.229642
4	0.243212	0.253225	0.242149	0.249124	0.199977	0.253828	0.279164

In [33]:

```

'''
코드는 메타 모델로 로지스틱 회귀를 사용하여 앙상블 모델의 성능을 평가합니다.
구체적으로, 교차 검증을 통해 메타 모델의 ROC AUC 점수를 계산하고, 그 평균을 구합니다.

# 1. 메타 모델 설정
로지스틱 회귀 모델을 메타 모델로 설정합니다.
- `LogisticRegression(solver='lbfgs')`: 로지스틱 회귀 모델을 생성합니다.
  `solver='lbfgs'`는 최적화 알고리즘으로 'lbfgs'를 사용합니다.

```

2. 메타 모델 입력 특성 선택

메타 모델에 사용할 입력 특성을 지정합니다.

- ``X_stk``: 메타 모델의 입력 특성으로 사용할 예측 확률 열들의 리스트입니다.
이 열들은 이전에 생성된 ``df_cv`` 데이터프레임에 포함되어 있습니다.

3. 교차 검증을 통한 성능 평가

교차 검증을 통해 메타 모델의 ROC AUC 점수를 계산하고, 그 평균을 구합니다.

내부 동작 설명

- ``cross_val_score``: 교차 검증을 통해 모델의 성능을 평가합니다.
- ``clf_meta_lr``: 메타 모델인 로지스틱 회귀 모델입니다.
- ``df_cv[X_stk]``: 메타 모델의 입력 특성 데이터입니다.
이전 단계에서 생성된 예측 확률 데이터프레임에서 ``X_stk``에 해당하는 열들만 선택합니다.
- ``df_train['failure']``: 타겟 변수입니다.
- ``groups=df_train['product_code']``: 그룹 정보를 사용하여 그룹별로 교차 검증을 수행합니다.
이는 동일한 제품 코드에 속하는 샘플이 동일한 폴드에 속하게 합니다.
- ``scoring='roc_auc'``: ROC AUC 점수를 성능 평가 지표로 사용합니다.
- ``cv=gcv``: 교차 검증 분할 전략입니다. ``gcv``는 GroupKFold 등의 그룹 기반 교차 검증 객체일 가능성이 높습니다.

평균 ROC AUC 점수 계산

- ``np.mean(...)``: 교차 검증의 결과로 얻은 ROC AUC 점수들의 평균을 계산합니다.

요약

이 코드는 다음과 같은 작업을 수행합니다:

1. 메타 모델로 로지스틱 회귀 모델을 설정합니다.
2. 메타 모델의 입력 특성으로 사용할 예측 확률 열들을 선택합니다.
3. 교차 검증을 통해 메타 모델의 ROC AUC 점수를 계산하고, 그 평균을 구합니다.

결과적으로, 메타 모델의 성능을 ROC AUC 점수로 평가하여 평균 점수를 출력합니다.

이 점수는 메타 모델이 얼마나 잘 작동하는지를 나타내며, 1에 가까울수록 더 좋은 성능을 의미합니다.

...

메타 모델을 검증해봅니다.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

```
clf_meta_lr = LogisticRegression(solver='lbfgs')
X_stk = ['baseline', 'lr2', 'lr3', 'lda', 'rf', 'rf2', 'xgb']
```

```
np.mean(
    cross_val_score(
        clf_meta_lr, df_cv[X_stk], df_train['failure'],
```

```

        groups=df_train['product_code'], scoring='roc_auc', cv=gcv
    )
)

```

Out[33]: 0.5900542516283906

In [34]: '''

코드는 앙상블 모델의 메타 모델을 학습시키고, 테스트 데이터에 대한 최종 예측을 수행하여 ROC AUC 점수를 계산하고 출력하는 과정을 수행합니다.

1. 테스트 데이터에 대한 예측 확률 계산

`df\_stk\_test`는 각 모델을 학습시키고 테스트 데이터에 대한 예측 확률을 계산한 결과입니다.

내부 동작 설명

- `clf.fit(df\_train[X\_all], df\_train['failure'])`:

각 모델을 훈련 데이터(`df\_train`)로 학습시킵니다.

`X\_all`은 모든 피처를 나타내며, `'failure'`는 타겟 변수입니다.

- `.predict\_proba(df\_test[X\_all])[:, 1]`:

학습된 모델을 사용하여 테스트 데이터(`df\_test`)에 대한 예측 확률을 계산합니다.

`predict\_proba`는 각 클래스에 대한 확률을 반환하며, `[:, 1]`은 양성 클래스(1)에 대한 확률을 선택합니다.

- `pd.Series(..., index=df\_test.index, name=model\_name)`:

예측 확률을 시리즈로 변환합니다.

- `index=df\_test.index`: 테스트 데이터의 인덱스를 유지합니다.

- `name=model\_name`: 시리즈의 이름을 모델의 이름으로 설정합니다.

리스트 컴프리헨션

리스트 컴프리헨션을 사용하여 위의 작업을 `model\_list`에 있는 모든 모델에 대해 반복합니다.

데이터프레임 결합

`pd.concat(..., axis=1)`을 사용하여 각 모델의 예측 확률 시리즈를 열 방향으로 결합하여 하나의 데이터프레임을 만듭니다. 결과적으로 `df\_stk\_test` 데이터프레임에는 각 모델의 예측 확률이 열로 추가됩니다.

2. 메타 모델 학습 및 예측

메타 모델을 학습시키고, 테스트 데이터에 대한 예측을 수행합니다.

내부 동작 설명

- `clf\_meta\_lr.fit(df\_cv[X\_stk], df\_train['failure'])`: 이전에 생성된 교차 검증 결과(`df\_cv`)를 사용하여 메타 모델을 학습시킵니다. `X\_stk`는 메타 모델의 입력 특성입니다.

- `.predict\_proba(df\_stk\_test[X\_stk])[:, 1]`: 학습된 메타 모델을 사용하여 테스트 데이터에 대한 예측 확률을 계산합니다.

`predict\_proba`는 각 클래스에 대한 확률을 반환하며, `[:, 1]`은 양성 클래스(1)에 대한 확률을 선택합니다.

3. ROC AUC 점수 계산 및 출력

최종 예측 결과와 실제 값을 비교하여 ROC AUC 점수를 계산하고 출력합니다.

- `roc\_auc\_score(df\_ans['failure'], prd)`:

```

실제 타겟 변수(df_ans['failure'])와 예측 확률(prd)을 비교하여 ROC AUC 점수를 계산합니다.
- print("Baseline 채점 결과:", roc_auc_score(df_ans['failure'], prd)): 계산된 ROC AUC 점수를 출력합니다.

# 요약
이 코드는 다음과 같은 작업을 수행합니다:
1. 여러 모델을 사용하여 테스트 데이터에 대한 예측 확률을 계산합니다.
2. 메타 모델로 로지스틱 회귀 모델을 학습시킵니다.
3. 메타 모델을 사용하여 테스트 데이터에 대한 최종 예측을 수행합니다.
4. 예측 결과를 실제 값과 비교하여 ROC AUC 점수를 계산하고 출력합니다.

결과적으로, 메타 모델의 성능을 ROC AUC 점수로 평가하여 그 결과를 출력합니다.
이 점수는 메타 모델이 테스트 데이터에서 얼마나 잘 작동하는지를 나타내며, 1에 가까울수록 더 좋은 성능을 의미합니다.
'''

# 2단계 메타 모델을 학습 시키고, 예측해 봅니다.

# 테스트 메타 모델 데이터를 만듭니다.
df_stk_test = pd.concat([
    pd.Series(
        clf.fit(df_train[X_all], df_train['failure']).predict_proba(df_test[X_all])[:, 1],
        index=df_test.index, name=model_name
    )
    for model_name, clf in model_list
], axis=1)
display(df_stk_test.head())

# 메타 모델을 학습 시키고, 테스트 메타 모델 데이터로 예측을 합니다.
prd = clf_meta_lr.fit(df_cv[X_stk], df_train['failure']).predict_proba(df_stk_test[X_stk])[:, 1]
print("Baseline 채점 결과:", roc_auc_score(df_ans['failure'], prd))

```

	baseline	lr2	lr3	lda	rf	rf2	xgb
id							
16115	0.145254	0.158406	0.132483	0.140965	0.131637	0.139301	0.188771
16116	0.178405	0.169721	0.175762	0.174935	0.169701	0.182337	0.214275
16117	0.215412	0.214998	0.222290	0.214288	0.205623	0.206804	0.238981
16118	0.263162	0.267503	0.266052	0.267748	0.275081	0.233135	0.300727
16119	0.215025	0.207329	0.220705	0.213243	0.203600	0.220509	0.237953

Baseline 채점 결과: 0.5914125449640287

In []: